

Prototype 4

Team

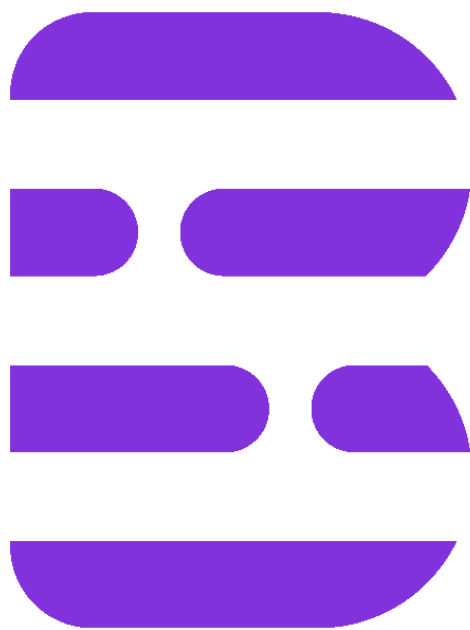
Name: 1C Team Members:

Full Name	GitHub Profile
Andrés Felipe Alarcón Pulido	andrefalar
Juan Jerónimo Gómez Rubiano	jujgomezru
Diego Esteban Ospina Ladino	DOspinalUN23
Jared Mijail Ramírez Escalante	JaredMijailRE
Felipe Rojas Marín	Olyveon
Juan Camilo Rosero Santisteban	juan-camilo-rosero

Software System

Name: Blume

Logo:



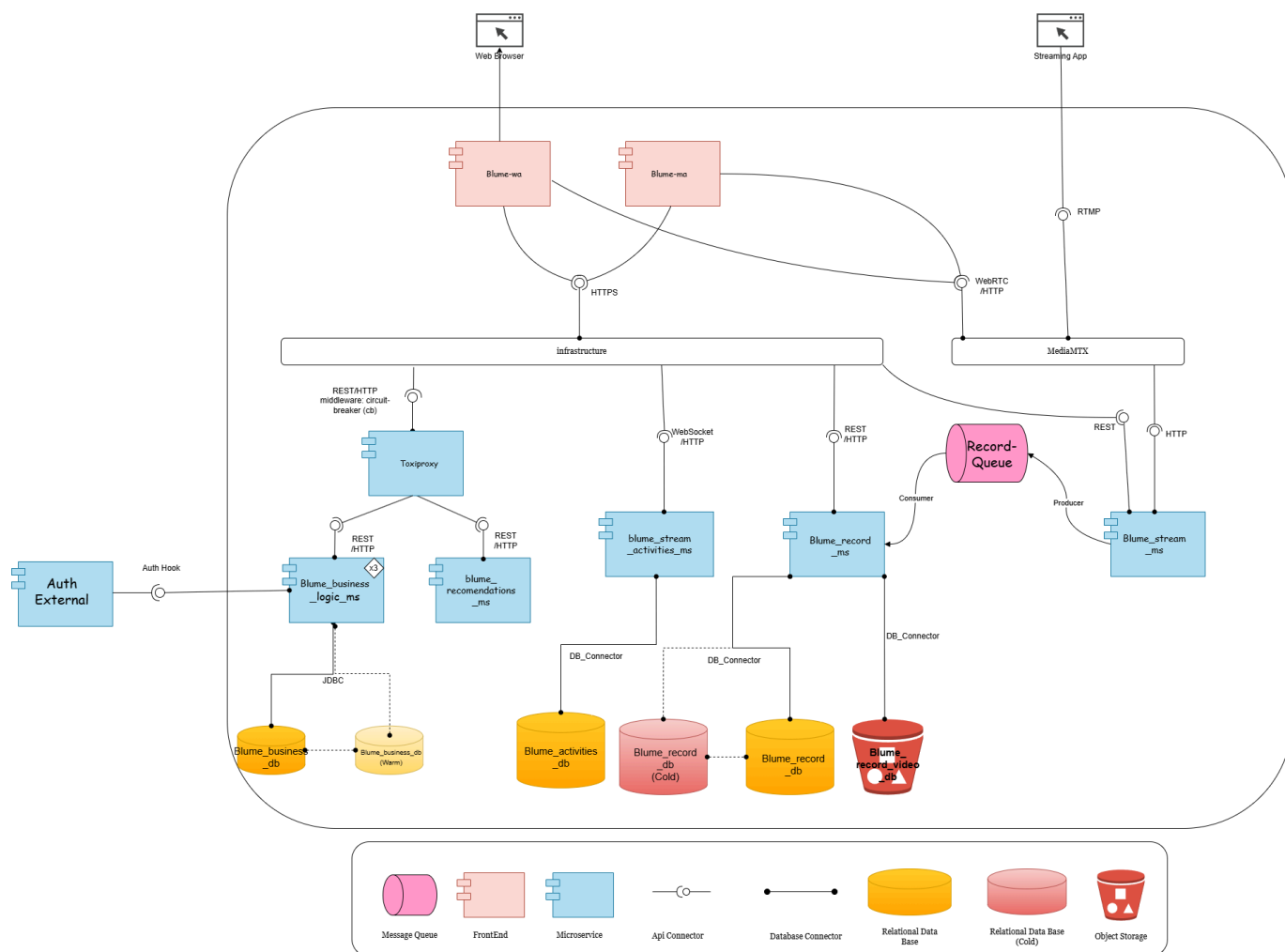
Description

Blume is a streaming and learning platform composed of microservices. It allows user authentication, creation and use of channels, grading students live-streaming from OBS, WebRTC playback in a browser, and management of historical recordings.

Architectural Structures

Component-and-Connector (C&C) Structure

• View:



Description of architectural elements and relations

Component	Type	Description	Relationships (origin → destiny · connector)
Web Browser	External Client	User that access the system from a web platform.	→ blume-wa · user interaction (UI)
Mobile client	External Client	User that access the system from the native mobile app.	→ blume-ma · user interaction (UI)
Streaming App	External Client	Recording app (i.e. OBS) that sends the live video stream.	→ MediaMTX · RTMP

blume-wa	FrontEnd	Blume's web client (Next.js). Pages, session state, API requesting, and live playback.	→ Blume_ag · REST; → MediaMTX · WebRTC/HTTP
blume-ma	FrontEnd	Blume's mobile client (Flutter). Same API and live playback as in web.	→ Blume_ag · REST; → MediaMTX · WebRTC/HTTP
Blume_ag	API Gateway	HTTP single entrance point. Routes petitions from the front-ends to the different service backends.	← blume-wa, blume-ma · REST; → blume_business_logic_ms, blume_recommendation_ms, blume_record_ms, blume_stream_ms · REST; → blume_stream_activities_ms · WebSocket; ← MediaMTX · REST
MediaMTX	Media Server	Media server: RTMP ingest, WebRTC/WHEP playback and recording fragments generation.	← Streaming App · RTMP; ← blume-wa, blume-ma · WebRTC/HTTP (proxied via infrastructure); → blume_stream_ms · HTTP (stream key auth hook on publish; recording segment-complete notification)
Auth External	External service	External authentication provider (i. e. Firebase / Google Identity).	→ blume_business_logic_ms · Auth Hook
blume_business_logic_ms	Microservice	Main Business Logic: users, channels, streams, local and via external provider authentication, JWT y core domain.	← Blume_ag · REST; ← Auth External · Auth Hook; → blume_business_db · JDBC
blume_stream_activities_ms	Microservice Cluster	A decentralized cluster of Elixir/Phoenix nodes for live interactions. It operates as a distributed mesh sharing real-time state.	← Blume_ag · WebSocket; → blume_stream_activities_db · database connector; Peer Nodes · **Erlang Distribution Protocol (EDP), Port 9000
**			

blume_recommendations_ms	Microservice	Class/stream recommendations according to other services and scoring.	← infrastructure · REST ; → ToxiProxy · HTTP (stream and channel data queries, proxied to blume_business_logic_ms); → blume_stream_activities_ms · HTTP (per-stream engagement data)
blume_stream_ms	Microservice	Streaming orchestration: publish/read authorization, viewer session, hooks with MediaMTX and recording publishing.	← Blume_ag · REST ; ↔ MediaMTX · REST , http ; → Record-Queue · Producer
blume_record_ms	Microservice	Asynchronous recording processing: listens to events, uploads video to object storage and persists metadata.	← Blume_ag · REST ; ← Record-Queue · Consumer ; → blume_record_db · database connector; → blume_record_video_db · storage connector
blume_business_db	Relational Database	Business data persistence: users, roles, channels, streams and main domain entities.	← blume_business_logic_ms · JDBC
blume_stream_activities_db	Relational Database	Persistence of chat messages, viewer sessions and live activities.	← blume_stream_activities_ms · database connector
blume_record_db	Relational Database	Recording metadata (identifiers, routes, status, object reference).	← blume_record_ms · database connector
blume_record_video_db	Object Storage	Storage of video files from processed recordings.	← blume_record_ms · Storage connector

Record-Queue	Message Queue	Asynchronous queue between the stream engine and the recording service (decouples notifications and processing).	← blume_stream_ms · Producer ; → blume_record_ms · Consumer
ToxiProxy	Proxy Component	Transparent TCP/HTTP proxy permanently interposed between blume_ag and blume_business_logic_ms on all /api traffic. Also sits between blume_recomendations_ms and blume_business_logic_ms. Its secondary capability — controllable fault injection (latency, connection resets) via its management API — enables empirical validation of the circuit-breaker policy without modifying any application service.	← blume_ag · HTTP (all /api requests forwarded by <i>Traefik</i>); ← blume_recomendations_ms · HTTP (stream data queries); → blume_business_logic_ms · HTTP (proxied requests, with optional injected)

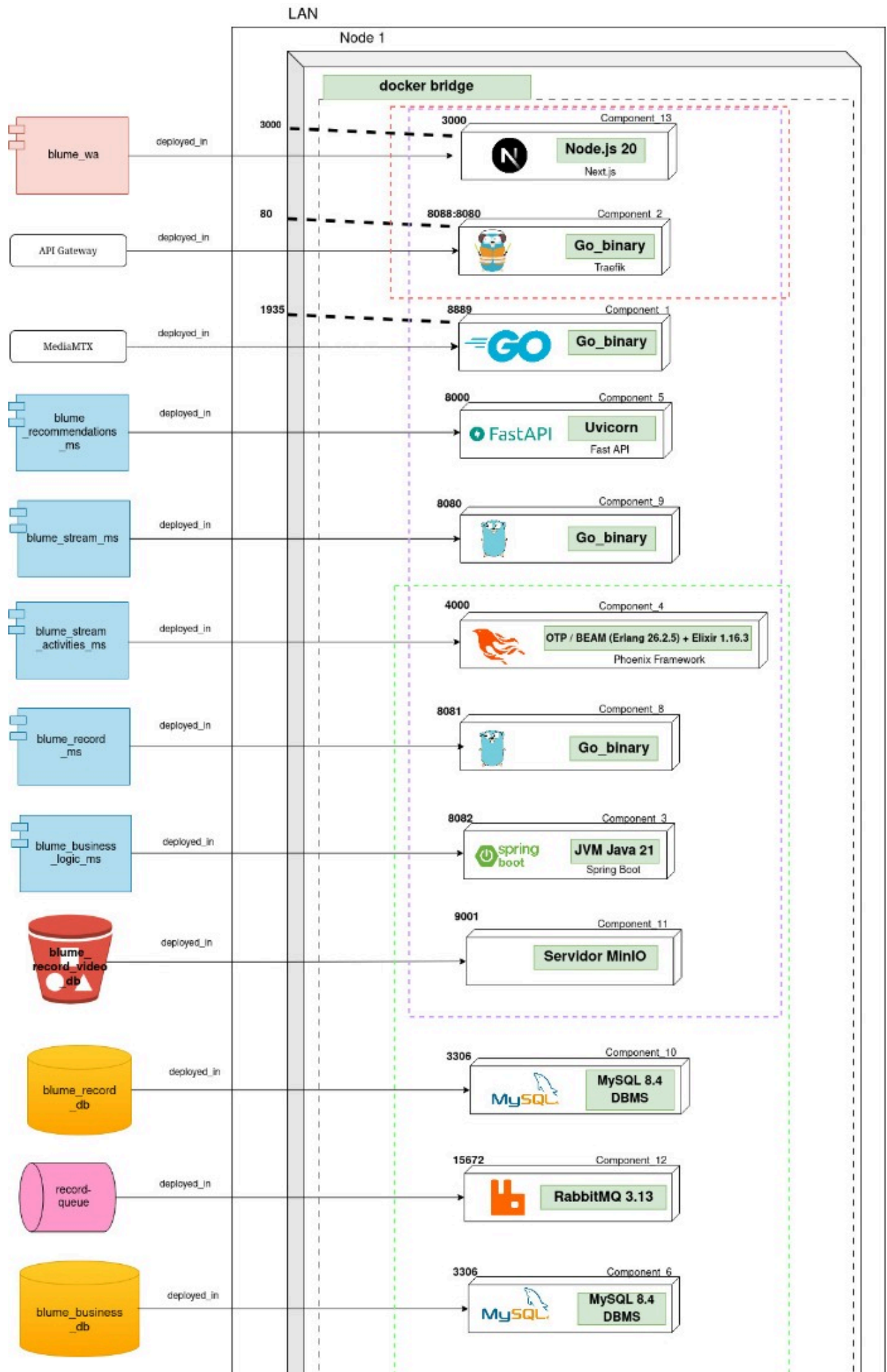
Description of architectural styles and patterns used

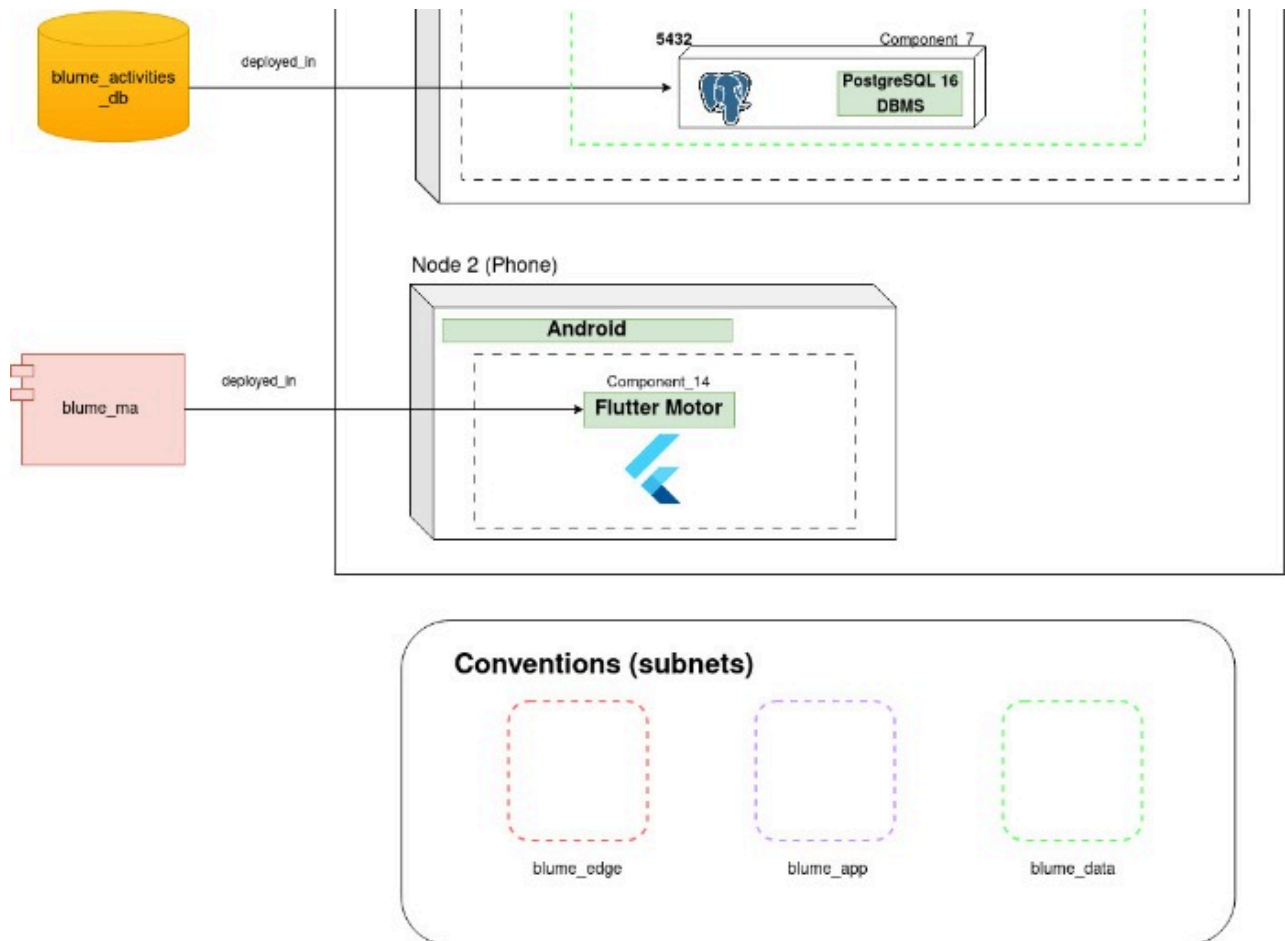
- API Gateway:** Within the framework of the Components and Connectors (C&C) view, the **API Gateway** pattern is implemented through the `Blume_ag` component, acting as a centralized gateway and single point of entry for external clients (`blume_wa` and `blume_ma`). As evidenced in the diagram `DiagramsDelivery_1-CyCView.drawio.png` , this architectural style functions as a routing connector and a mediator that encapsulates the internal topology of the system, isolating clients from the complexity of the underlying microservices network. The API Gateway assumes the responsibility for protocol adaptation, channeling synchronous REST requests towards the different microservices, for example, managing continuous bidirectional connections (WebSocket) towards `blume_stream_activities_ms` . Its inclusion drastically reduces the coupling between the frontend and the backend, mitigating network overhead by avoiding "chatty" communication (multiple direct calls from clients to different services) and establishing a cohesive layer for perimeter access control and routing. As a refinement of this centralized routing, the gateway's connector toward `blume_business_logic_ms` is not a simple call-return link. Traffic on the `/api` path is first subject to a circuit-breaker policy (business-cb middleware) embedded within *Traefik* itself, which monitors error rates on the connector and interrupts forwarding when thresholds are breached — reducing this connector to a resilience-enriched call-return connector with open/half-open/closed states. Downstream of the gateway, a dedicated proxy component (*ToxiProxy*) acts as the actual upstream target, receiving requests from both *Traefik* and `blume_recomendations_ms` , and forwarding them to the business logic replicas. *ToxiProxy* is a standalone runtime component — not a behavior of *Traefik* — whose role is to enable controlled fault injection for resilience validation of the circuit-breaker mechanism itself.
 Asynchronous Messaging (Message Broker / AMQP): To address background processing needs, the architecture adopts an **Asynchronous Messaging** style mediated by the `Record-Queue` component (represented as a Message Queue in the diagram). In C&C semantics, this pattern employs message-oriented asynchronous data distribution connectors that establish a producer-consumer communication between `blume_stream_ms` (Producer) and `blume_record_ms` (Consumer). Theoretically, the use of a message queue guarantees the temporal and spatial decoupling of components: the streaming engine can notify the completion of a recording segment without needing to wait for a response, while the recording service processes the load in a deferred manner. This pattern significantly increases the system's resilience and fault tolerance, as it acts as a buffer that absorbs peak loads, preventing costly I/O operations (such as video persistence in object storage) from degrading the critical performance of the live stream.
 Synchronous Communication (HTTP/REST) For interactive transactions that demand determinism and immediate responses, the system is grounded in a **Synchronous Communication** style based on REST (Representational State Transfer) principles over HTTP. From the C&C perspective, this pattern materializes through stateless call-return connectors that backbone both the clients' interaction with `Blume_ag` and the internal orchestration from the Gateway to the core microservices (`blume_business_logic_ms` , `blume_recommendation_ms` , etc.) and the media server (*MediaMTX*). At a theoretical level, the stateless constraint of the connector ensures that each request contains all the context necessary to be processed, which is vital for enabling the horizontal scalability of the microservices. This style promotes a standardized semantics based on resources and verbs, facilitating interoperability and guaranteeing a robust request-response model for the synchronous operations of the business domain.
 Cluster Pattern (Peer-to-Peer) Applied internally within the Live Stream Activities component. It uses Erlang's native distribution capabilities to form a mesh network of nodes,

allowing high availability and fault tolerance for WebSocket connections without a single point of failure in the business logic. State synchronization and PubSub messages are broadcasted across the cluster via the Erlang Distribution Protocol over a Tailscale VPN overlay.

Deployment Structure

• **View:**





Description of architectural elements and relations

The Blume platform's physical mapping is modeled through a distributed, multi-node allocation structure. It captures how computational execution environments, networking segments, and persistent artifacts are instantiated across hardware barriers.

The system topology is logically split into two distinct physical domains operating over a shared Local Area Network (LAN) connector, establishing a strict perimeter between the client presentation space and backend server processing:

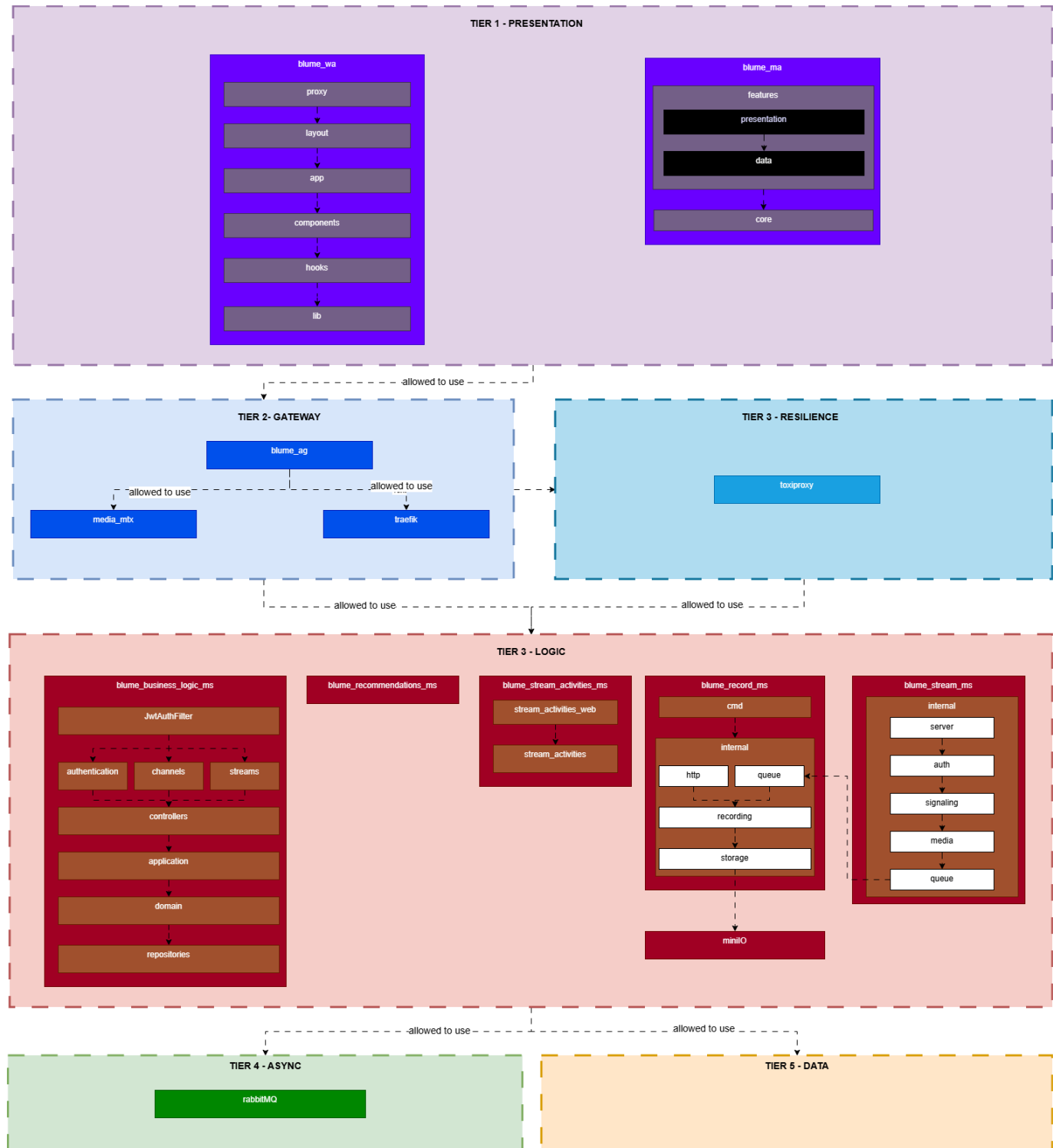
- **Node 1 (Backend Host Machine):** A high-compute environment executing an operating system instance that hosts a virtualization layer. It holds the core system container that executes the entire Blume ecosystem. All internal routing, messaging, media packaging, and persistent storage occur within this node's boundaries.
- **Node 2 (Client Phone Device):** A mobile hardware node running the Android Execution Environment. This node encapsulates the presentation client assets, specifically running the binary compiled by the Flutter Motor (Component 14). It acts purely as a consumer of edge-routed streams and REST schemas.
- **Multi-Node Cluster:** Composed of three nodes—two local and one in the cloud—this architecture establishes an extra layer of structural resilience (elaborated on in the QA section). While each node operates in a distinct network environment, they are securely interconnected into a single private domain through a shared VPN.

Description of architectural patterns used

- **Containerization (OS-Level Virtualization):** This pattern involves packaging an application and its entire runtime dependency tree into isolated user-space instances called containers, which share the host operating system's kernel. In Blume, this is utilized to containerize heterogeneous microservices (Spring Boot, FastAPI, Phoenix, Go), ensuring environmental parity across the system lifecycle and preventing dependency conflicts on the physical host machine (Node 1).

Layered Structure

View:



Architectural Elements and Relations

Elements

On the Layered structure, we can see the system is divided in 6 layers, which are as follows:

- **Presentation:** This tier is the only layer that users ever directly interact with. Its sole responsibility is rendering UI and translating user intent into structured requests for the layer below — it holds no business logic and owns no data.
 - **blume_wa** is a *Next.js 16* server-rendered web app. It delegates all auth, stream, and channel concerns to the backend via REST and receives JWT identity through an httpOnly cookie. It also connects a Phoenix WebSocket client for live chat/polls.

- `blume_ma` is a *Flutter* cross-platform mobile client. It follows Clean Architecture internally (Presentation → Domain → Data), but from the system's perspective it is purely a consumer: it reads from REST endpoints, plays HLS from *MediaMTX*, and relies on the same cookie-based auth contract.

Neither client has its own database, business rules, or message queue access. They are only concerned with user experience and interaction, which includes defining the aesthetics and application flow.

- **Gateway:** This tier is the system's single entry point and traffic director. *Traefik* owns HTTP/HTTPS traffic; *MediaMTX* owns the media plane (RTMP/HLS/WebRTC).

Its responsibilities are:

- Routing requests to the correct microservice by path prefix and priority (e.g., `/api/v1` → recommendations, `/socket` → Phoenix, `/api/recordings` → record-ms, `/` → Next.js).
- TLS termination — all external traffic arrives over HTTPS; internal traffic flows over the private `blume_app` Docker network.
- Network isolation — enforces a three-zone boundary: `blume_edge` (DMZ, reachable from outside), `blume_app` (services, internal only), `blume_data` (databases/queues, never exposed to the host).

This tier does not transform data or execute logic. It is a pure structural layer: it defines who can talk to whom and how traffic enters the system. It also holds the main container that executes the entire Blume system. Every other components' Dockerfile only deploys its own content.

- **Resilience:** Orthogonal to the synchronous communication style, the system applies a circuit-breaker pattern to the connector from *Traefik* to `blume_business_logic_ms`. This connector is a *Traefik* middleware, that monitors the error rate on the connector and transitions it between closed (normal), open (short-circuited), and half-open (probing) states, preventing cascading failures from propagating to clients. *ToxiProxy* is its standalone component. It's a proxy runtime element with its own container, network address, and connectors. It sits downstream of the circuit-breaker policy and upstream of `blume_business_logic_ms`, accepting connections from both *Traefik* and `blume_recommendations_ms`. Its architectural purpose is to allow controlled injection of network faults (latency, packet loss, connection resets) against the connector to `business_logic_ms`, enabling empirical validation that the circuit-breaker policy fires correctly under realistic failure conditions.
- **Logic:** This is the widest tier, where every meaningful computation in the system happens. Each microservice here owns a specific bounded context. Together they compose the entire practical functions of the system.
 - `blume_business_logic_ms` : Identity, auth (local + Firebase), channels, classes, access control, grades. The system's source of truth for domain entities. Internally designed with hexagonal architecture.
 - `blume_stream_ms` : Real-time streaming orchestration: viewer sessions, HLS management, RTMP authorization, SSE for viewer counts. Talks to *MediaMTX* and *RabbitMQ*.
 - `blume_stream_activities_ms` : Live engagement: chat, polls, quizzes over WebSocket (Phoenix Channels). Writes engagement data consumed by recommendations.
 - `blume_recommendations_ms` : Stateless scoring engine: aggregates stream metadata and engagement data from other services, ranks results via hybrid algorithm (recency + engagement + affinity). No DB of its own.
 - `blume_record_ms` : Post-processing pipeline: detects when *MediaMTX* finishes a recording, uploads it to MinIO/S3, tracks metadata in its own MySQL instance.

These services communicate synchronously (REST over `blume_app`) and asynchronously (via *RabbitMQ*). They are the only tier allowed to mutate the system's state.

- **Async:** This tier decouples the services within Tier 3 from each other on the time axis. Its purpose is to allow high-latency or best-effort operations to happen without blocking synchronous request paths. In practice it carries exactly one pipeline: when a live stream ends, *MediaMTX* fires an HTTP webhook to `blume_stream_ms` → `blume_stream_ms` publishes a message to the `recordings.ready` queue → `blume_record_ms` consumes it and begins the upload-to-S3 process. Neither service needs to wait for the other; the recording pipeline is fully decoupled from stream teardown.

This tier has no logic of its own — it is pure message routing and buffering. If it goes down, streams still run; recordings are delayed, not lost (reconciliation loop in `blume_record_ms` also polls the filesystem as a fallback).

- **Data:** This tier stores all durable state in the system. It is partitioned by service ownership — no two services share a database, which is a key microservices principle:

Store	Owner	What it holds
MySQL (blume_db)	blume_business_logic_ms	Users, channels, classes, enrollments, grades, Flyway migrations
MySQL (recordings_mysql)	blume_record_ms	Recording metadata (stream key, S3 URL, timestamps)
PostgreSQL (postgres_activities)	blume_stream_activities_ms	Chat messages, polls, quiz responses, engagement events
MinIO (S3-compatible)	blume_record_ms	Raw video files (.mp4) uploaded from MediaMTX recordings

All four stores live on the blume_data network, which has no host-level port exposure — they are reachable only by Tier 3 services through the internal Docker network.

Relations

The *allowed to use* relation is an architectural constraint on dependency direction, not just a description of runtime communication. It indicates that, the Layer that is allowed to use, Layer "A", is allowed to know about, call, and depend on another Layer "B". This means, the Layer "B" in this scenario is completely ignorant of Layer "A". This has two consequences:

1. Dependencies are one-directional. If Presentation is allowed to use Communication, it means Presentation can initiate calls to Communication — but Communication must never initiate calls up to Presentation. A lower layer should never need to know that an upper layer exists.
2. It defines who is responsible for stability. The layer being "used" must offer a stable interface, because the layer above depends on it. Changes to a lower layer can force adaptation above; changes to an upper layer should never break anything below.

Blume uses relaxed layering. This means, that it's possible for an upper layer to use multiple layers beneath them, not just one. This can be seen specifically by the Logic Layer, which talks directly to both Async and Persistence, rather than having to use one of them to complete a flux (which makes sense since their both beneath Logic, but hold different responsibilities). On the otherhand, both Async and Persistence are lower layers and do not know about Logic or each other, they only provide their services.

- **Presentation → Gateway:** This relation has two distinct channels depending on what kind of traffic is involved.
 - HTTP/REST and WebSocket — via *Traefik*: Both `blume_wa` and `blume_ma` send all API calls to *Traefik* as their sole known host. The clients have no knowledge of which service sits behind any given path — they only know the gateway's address. The JWT session identity travels as an `httpOnly` cookie on every request. For WebSocket, `blume_wa` upgrades the connection at `/socket` through *Traefik*, which proxies it to Phoenix. From the client's perspective it is a single persistent connection; *Traefik* is transparent.
 - HLS media — via *MediaMTX*: When a client wants to watch a live stream, it first calls `stream_ms` through *Traefik* to get a viewer session (this returns a WebRTC/WHIP manifest URL pointing at *MediaMTX* port 8889). From that point on, the client connects directly to *MediaMTX* for the actual video data. `blume_ma` uses `video_player` + `Chewie` for this; `blume_wa` uses a browser-native video player. Presentation uses *MediaMTX* as a pure media delivery endpoint — it has no awareness of what authentication decision was made upstream.
 - RTMP — professor's OBS → *MediaMTX*: When a professor starts streaming, their OBS client pushes an RTMP stream directly to *MediaMTX* on port 1935 using a stream key obtained from the business logic service.

This is the only incoming flow that bypasses *Traefik* entirely — RTMP is not HTTP and *Traefik* does not handle it.

- **Gateway → Logic** This is where the gateway tier delegates all decisions it cannot make itself.
 - *Traefik* → Logic services (HTTP proxy): *Traefik* holds no logic. It matches the incoming request path against its routing table, strips the prefix if configured, and forwards the full HTTP request (headers, cookies, body) to the target service on the blume_app network. The Logic service sees the original request as if it arrived directly. The routing decision is purely structural: `/api` → `ToxiProxy:38082` → `business_logic_ms:8082` (the `business-proxy` *Traefik* service always targets `ToxiProxy` first), `/api/v1` → `recomendations_ms:8000`, `/socket` → `stream_activities_ms:4000`, `/api/recordings` → `record_ms:8081`, and so on. Priority values prevent ambiguous path overlaps.
 - *MediaMTX* → `stream_ms`: When an RTMP stream arrives at *MediaMTX*, *MediaMTX* does not decide whether to accept it. Instead it sends a synchronous HTTP callback to `stream_ms` at `/auth/mediamtx`, passing the stream key. `stream_ms` checks whether the key belongs to a valid, authorized class and returns 200 (allow) or 403 (deny). *MediaMTX* then accepts or drops the stream accordingly. *MediaMTX* knows nothing about users, channels, or classes — it delegates the entire authorization decision to the Logic tier and acts on the binary response.
- **Logic → Logic**: These are service-to-service HTTP calls on the blume_app network, bypassing *Traefik* entirely. They are same-tier dependencies and the most architecturally significant coupling in the system.
 - `recomendations_ms` → `ToxiProxy` → `business_logic_ms`: `recomendations_ms` calls `GET /api/clases` to fetch stream and channel metadata as input to its scoring algorithm. This call is not made directly to `business_logic_ms` — `STREAMS_API_URL` in `docker-compose` points to `http://toxiproxy:38082`, so the request crosses into the Resilience tier before reaching the Logic tier. This makes `recomendations_ms` dependent on both `ToxiProxy` and `business_logic_ms`: if either is unavailable, the recommendation engine degrades (the in-memory cache with a 2-minute

TTL acts as a buffer).

```
- `recomendations_ms` → `stream_activities_ms`: `recomendations_ms` calls `GET
```

`/api/analytics/streams/engagement` on the Phoenix service to retrieve per-stream engagement counts (chat messages, polls, quizzes). These feed the engagement component of the scoring formula. This call is made directly to `blume_stream_activities_ms:4000`, not through *ToxiProxy*.

- **Logic → Async**: This relation is unidirectional at the dependency level but bidirectional at the message level. The Logic tier owns the queue interaction — *RabbitMQ* is passive.
 - `stream_ms` → *RabbitMQ* (publisher flow): When *MediaMTX* signals that a stream has finished publishing, `stream_ms` places a message on the `recordings.ready` queue. The message carries enough context for a consumer to locate and process the recording (stream key, path). `stream_ms` does not know or care who will consume this message.
 - `record_ms` → *RabbitMQ* (consumer flow): `record_ms` subscribes to the same queue and processes each message as it arrives: it locates the recording file on the shared volume, waits for the file to stabilize (no more writes), uploads it to MinIO, and stores the resulting metadata. `record_ms` also runs a reconciliation loop that scans the filesystem independently — so if *RabbitMQ* is unavailable, recordings are eventually processed anyway.

The Async tier never calls into Logic. Messages flow from Logic into the queue and back out to Logic. The dependency arrow (who knows about whom) still points downward: Logic knows about *RabbitMQ*'s queue names and AMQP protocol; *RabbitMQ* knows nothing about Logic.

- **Logic → Persistence**: Each service owns exactly one store and accesses it directly. No service queries another service's database — this is the microservices database-per-service rule enforced at the network level (`blume_data` is only reachable from `blume_app` services).

- `business_logic_ms` → MySQL (`blume_db`): Uses Spring Data JPA for all reads and writes. Flyway manages the schema — DDL is never applied at runtime, only through versioned migration files. This store holds all primary domain entities: users, channels, classes,

enrollments, grades.

```
- `record_ms` → MySQL (recordings_mysql): Uses direct SQL (no ORM). After a successful S3 upload,
record_ms writes the recording's metadata (stream key, S3 URL timestamps) into this separate database.
It is isolated from `blume_db` — `business_logic_ms` never touches it.

- `stream_activities_ms` → PostgreSQL (postgres_activities): Uses Ecto for all queries and changesets.
Stores the live engagement data: chat messages, poll definitions and responses, quiz events. This is
the data recommendations_ms aggregates.

- `record_ms` → MinIO: Uses the S3-compatible PutObject API. MinIO is technically an object store
rather than a relational database, but it
```

lives in the Persistence tier because it is a durable, append-only store of binary state. `record_ms` writes .mp4 files here and stores the resulting URL in MySQL. No other service writes to MinIO; `blume_wa/blume_ma` read recordings by receiving URLs from `record_ms` via *Traefik*.

Description of architectural patterns used

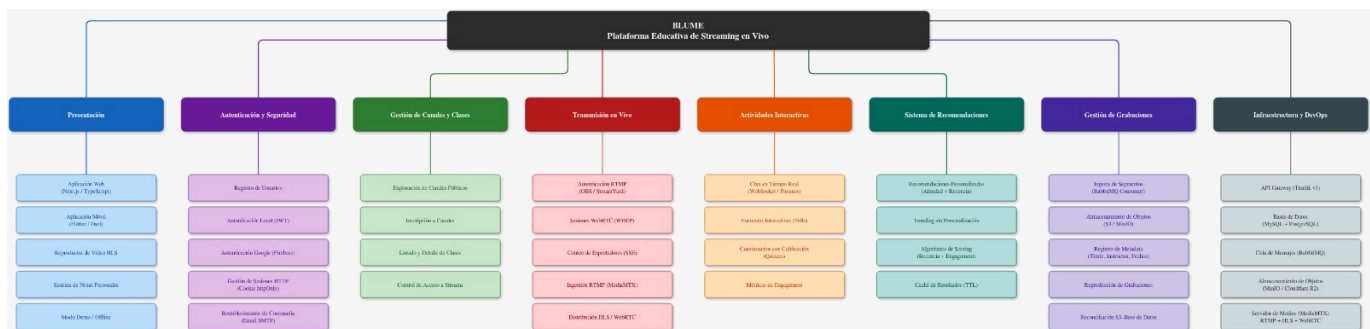
- **Model-View-Controller:** The MVC pattern is a way of organizing services through three components: Model, View, and Controller. The Model is the internal representation of data. It defines its structure, business logic, and internal rules. In this diagram, each microservice defines its own model. For example, in `blume_business_logic_ms`, models are found in repositories. Views are the way this information is presented; the key idea is that the same model can be represented in multiple ways, depending solely on content and not on internal logic. In our MVC implementation, Views are handled by `blume_wa`, such as the different React pages that make up the frontend. Controllers are software components responsible for accepting input and forwarding it in an interpretable form to the target component, without either communicating component needing to know about the other, making inter-component communication more flexible. In `blume_business_logic_ms`, this is handled by the controllers, which are located on the `infrastructure/adapters/in` subfolders of each domain.
- **Model-View-ViewModel:** An alternative to the MVC pattern. Unlike MVC, MVVM uses a ViewModel as an intermediary instead of a Controller. The ViewModel differs from a Controller in two ways. First, the ViewModel does not receive requests or execute responses the way a Controller does. Second, the ViewModel is an abstraction of the Model for the View, adapting data and exposing commands. The View observes the ViewModel to render state changes autonomously without a traditional controller. Being an abstraction, it can hold system state, whereas a Controller always behaves the same way as defined by its source code. In Blume, this pattern is used for handling mobile requests. The `blume_ma` frontend also connects to `Blume_ag` and therefore to the backend, but while `blume_wa` has an internal server (*Next.js*) that listens for and executes requests, *Flutter* is a constant execution environment. Views render themselves in response to state changes, orchestrated directly by *Flutter*, so no controller is needed.
- **Public vs. Published Interfaces:** This pattern consists of distinguishing between public and published interfaces and managing them accordingly. A public interface is one that is visible outside a class at the code level; since it's possible to control all its callers, it can be freely modified through refactoring. A published interface, on the other hand, is one explicitly exposed to external consumers outside of their control, making them significantly harder to change, because modifying them break code that is not accessible. In our system, the communication between microservices and the API Gateway functions as a published interface: the API Gateway has no access to the microservices' internal classes, only to their interfaces, and changing those interfaces directly impacts dependent logic. The advantage of designing around published interfaces is that they enforce a stable, well-defined contract. To keep this contract manageable, interfaces should be kept thin. In this case, it's implemented as a minimalist JSON files that provide only the information each service needs to respond to requests.
- **Data Access Object:** A DAO is an abstract interface applied to a database element. Basically, an internal, code-level representation of database contents that facilitates their processing by the system's logical components. The advantage of this pattern is that it allows representing the system's persistence layer without exposing the

underlying database's implementation details. This pattern is used extensively in Blume, primarily in `blume_business_logic_ms`. This component defines JPA Entities. These are data structures that mirror database contents. As well as Repositories, which are the Java-language expression of database queries applied over those entities. Additionally, `blume_business_logic_ms` applies Hexagonal Architecture (Ports and Adapters): Ports are the interfaces that define the contract between the application core and the outside world. Adapters translate and map data between external formats (JPA entities) and internal domain objects, ensuring the core business logic remains independent of persistence mechanisms. This means that classes implementing the ports depend neither on the database nor on the underlying logic. The DAO pattern is also used in `blume_record_ms` via repositories and interfaces, and in `blume_wa` to interact with the Next.js server.

- **Cache-Aside (Memoization):** A caching pattern in which the application itself is responsible for loading data into the cache on demand. On the first request, data is fetched from the original source and stored in an in-memory structure; subsequent requests retrieve the cached value directly, bypassing redundant computation or network calls. This pattern is applied in `blume_recomendations_ms`, which defines a dedicated cache class at `app/core/cache.py` that stores HTTP responses obtained from other microservices, specifying how the map is stored along with its associated read and write methods. Alongside it, `app/repositories/streams_repo.py` implements the lookup logic: before issuing a call to `blume_business_logic_ms` or `blume_stream_activities_ms`, it consults the cache first and only fetches from the source on a miss. This eliminates redundant inter-service calls and serves as the TTL-based buffer described in the Logic → Logic relation. *Hibernate* and *GORM* provide first-level (Identity Map) caching at the ORM layer, which is a distinct mechanism from the Cache-Aside pattern used here.
- **Inversion of Control and Dependency Injection:** Inversion of Control (IoC) is the broader principle by which the flow of control is inverted relative to conventional programming: instead of the developer's code calling a library when needed, a framework calls the developer's code at the appropriate time (the Hollywood Principle: "don't call us, we'll call you"). **Dependency Injection (DI)** is one specific form of IoC, in which a component declares the dependencies it needs rather than creating them internally, with an external assembler responsible for constructing and providing those dependencies. This is the form applied in `blume_record_ms`. The class `internal/recordings/service.go` does not instantiate its own dependencies; it declares what it needs. In this case, a Repository and an ObjectStorage. Then, `internal/main.go` constructs those dependencies and injects them. This makes the service decoupled from any specific implementation of its dependencies, improving testability and flexibility.

Decomposition Structure

View:



Description of architectural elements and relations

The BLUME system is decomposed into eight functional modules, each grouping a cohesive set of capabilities under a strict "is part of" relationship. No runtime communication or data flow is represented — this view describes structural ownership only.

- **Presentation** groups all user-facing components: the web application (*Next.js* / *TypeScript*), the mobile application (*Flutter* / *Dart*), the HLS video player, the personal notes system, and a demo/offline mode that operates without a live backend.

- **Authentication and Security** owns all identity and session management: local registration, JWT-based local authentication, Google authentication via Firebase, HTTP session management through HttpOnly cookies, and password reset via email (SMTP).
 - **Channel and Class Management** covers the academic content layer: public channel exploration, channel enrollment, class listing and detail, and stream access control.
 - **Live Streaming** handles the full live streaming pipeline: RTMP authentication for broadcasting tools (OBS / StreamYard), WebRTC session management (WHEP), real-time viewer counting via SSE, RTMP ingest through *MediaMTX*, and HLS / WebRTC distribution to viewers.
 - **Interactive Activities** groups all real-time engagement features running over Phoenix WebSockets: live chat, interactive polls, graded quizzes, and engagement metrics collection.
 - **Recommendation System** contains the content discovery engine: personalized recommendations (affinity + recency scoring), non-personalized trending, the hybrid scoring algorithm, and a TTL-based result cache to reduce redundant computation.
 - **Recording Management** owns the post-stream recording pipeline: segment ingestion via RabbitMQ consumer, object storage upload to S3/MinIO, metadata registration (title, instructor, timestamps), recording playback, and a reconciliation process that syncs S3 state with the database.
 - **Infrastructure and DevOps** contains all platform-level components: the API Gateway (Traefik v3), relational databases (MySQL + PostgreSQL), the message queue (RabbitMQ), object storage (MinIO / Cloudflare R2), and the media server (MediaMTX) handling RTMP ingest, HLS packaging, and WebRTC distribution.
-

Quality Attributes

Security

For this quality attribute we used 4 different patterns to overcome 4 different scenarios

Secure Channel

Scenario

- **Source:** A user enrolled as a student in a course that uses the Blume platform.
- **Stimulus:** They wish to modify the grades of the course in which they are enrolled. In this example, student Ana García Morales failed the 2nd Midterm of Computer Networks.
- **Artifact:** The HTTP connectors between clients and *Traefik*, specifically the session cookie (`blume_session`) transmitted on every authenticated request.

Unirse

▲
●
▼

Parcial 2 Examen 25% 2.9 [Calificar](#)

- ```
(base) jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infraestructure$ curl -X PUT http://localhost/api/canales/33f9bbb0-4d51-11f1-918-ce9288f2e7ef/notas/50 \
-H "Content-Type: application/json" \
-H "Cookie: blume_session=$PROF_TOKEN" \
-d '{"studentId": "33f6da31-4d51-11f1-9618-ce9288f2e7ef", "score": 5.0}'
{"data": null, "message": null}(base) jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infraestructure$
```

# Redes de Computadores

Canal del curso de Redes de Computadores — semestre 2026-1. Profesor: Fernando Reyes.

EN VIVO AHORA

● EN VIVO



## Sesión en vivo — Redes definidas por software (SDN)

Arquitectura SDN: plano de control vs. datos. OpenFlow y controladores como OpenDaylight.

Unirse

☰ Descripción    📺 Transmisiones pasadas    📎 Material de apoyo    📄 Notas

## Notas del curso

| Evaluación | Tipo   | Peso | Nota |
|------------|--------|------|------|
| Parcial 1  | Examen | 25%  | 4.0  |
| Parcial 2  | Examen | 25%  | 5.0  |

In this example, using `tcpdump`, at the moment the professor logs in on a public network, the student can access the traffic generated by the login event. In the initial state, the user's username and password are not sent directly, but a cookie (`blume_session`) is transmitted, which is used by the server to verify the user's identity. Once this cookie is obtained, along with other easily accessible information such as the user's internal class ID or the evaluation ID, a command can be sent to modify the grade — using the professor's cookie to impersonate them and submit a request to the backend. The web frontend confirms that the request successfully modified the grade, without the professor having made this change directly.

- **Response:** A TLS certificate is obtained and deployed to encrypt the system's network traffic.
- **Response Measure:** Basic network inspection tools such as `tcpdump` confirm that all intercepted traffic is encrypted, and the user cannot access the cookie required for authentication.

```
(base) jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$ sudo tcpdump -i br-673bda64ed6a -A -s 0 'tcp port 80' 2>/dev/null | grep -E 'Cookie:|Set-Cookie:|blume_session|HTTP/'
```

```
(base) jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$ sudo tcpdump -i br-673bda64ed6a -A -s 0 'tcp port 443' 2>/dev/null | strings | grep -E 'Cookie:|blume_session'
```

In our example, we run the same command after having implemented the HTTPS pattern and its corresponding CA certificates to protect the system's traffic. Running `tcpdump` no longer allows content to be viewed through ports 80 or 443, which are used for system communication. After implementing HTTPS, `tcpdump` only captures encrypted bytes and produces no interpretable output. Since the content cannot be interpreted, the cookie is transmitted securely, and the attacker cannot make any requests, as they have no way to impersonate the professor without their credentials.

- WireShark

The image displays a Wireshark packet capture of a TLS connection. The packet list on the left shows a sequence of packets from 192.168.80.175 to 192.168.80.177. Packet 2929 is highlighted, showing a TLSv1.3 record. The packet details pane on the right shows the structure of the TLSv1.3 record, including the Application Data (Hypertext Transfer Protocol) and the Encrypted Application Data. The packet bytes pane at the bottom shows the raw hex and ASCII data.

To further validate the effectiveness of the encryption mechanism, a network analysis was conducted using Wireshark. For this test, traffic was captured on a wireless network interface. A mobile device connected to the same local network was used to access and navigate the Blume web interface hosted on the deployment machine.

As captured in the log below, the network traffic was successfully intercepted. However, the packet details confirm that all payload data remains fully encrypted. The capture clearly identifies the source IP (the mobile device) and the destination IP (the host computer running the application), proving that while the communication path is visible, the sensitive contents—including session tokens and application data—are completely shielded from unauthorized inspection.

## Characteristics

- **Threat:** Malicious agent with network access
- **Attack:** Network traffic interception
- **Weakness:** With HTTP, information travels in plain text
- **Vulnerability:** An attacker could eavesdrop on traffic, compromising confidentiality
- **Risk:** Theft, exposure, or modification of sensitive information in transit
- **Countermeasure:** Implement HTTP/TLS to encrypt communication and protect data in transit

## Architectural Tactics

### Detect

For early threat detection, the following tactics are implemented:

- **Verify Message Integrity:** At the transport layer, TLS provides HMAC-based integrity verification, ensuring that messages cannot be tampered with in transit.

### Resist

To resist attacks, the following tactics are implemented:

- **Encryption:** The TLS certificate encrypts all HTTP connector traffic routed through *Traefik*. RTMP stream ingestion (port 1935) currently remains unencrypted, as RTMPS is not yet configured.
- **Actor Authentication:** Authentication relies on session cookies ( `blume_session` ) rather than sending raw credentials with every request. However, as the scenario demonstrates, the session cookie is itself equally sensitive: over plain HTTP it is transmitted in clear text and can be intercepted, which is precisely why TLS is required to protect it.
- **Limit Exposure:** Public access to sensitive ports is removed: port 15672 for RabbitMQ and ports 9000 and 9001 for MinIO. The Traefik API/dashboard is fully disabled via `--api.insecure=false` . Ports 8889 (MediaMTX WebRTC) and 1935 (RTMP ingestion) remain publicly exposed for operational reasons.
- **Change Default Configuration:** In addition to limiting exposure, default usernames and passwords for management dashboards were changed, in case an attacker is able to bypass the limited exposure barrier.

### React

The following tactics are implemented to respond promptly and efficiently to a successful attack:

- **Inform:** HSTS prevents browsers from sending requests over plain HTTP, protecting against downgrade attacks in which a man-in-the-middle could strip HTTPS and redirect traffic to plain HTTP.

### Recover

In the current implementation, the Secure Channel pattern does not handle the Recover portion of the Security Tactics.

## Architectural Pattern

The **Secure Channel** pattern consists of a design applied to a system's connectors — in this case using the HTTP protocol — over which an encryption algorithm is applied to protect the content of its traffic from external actors. In this case, the TLS protocol is used to encrypt the information transmitted by the previously implemented HTTP connectors. For this delivery, the pattern was correctly implemented in the local deployment of the system, on all HTTP connectors that use the *Traefik* API Gateway. It is implemented as follows:

The new version of `infrastructure/traefik/dynamic.yml` defines the use of the HTTPS protocol as follows:

```
routers:
 nextAuthClearSession:
 rule: "Path(`/api/auth/clear-session`)"
 entryPoints: ["websecure"]
 middlewares: ["hsts"]
 priority: 20
 service: blume_wa
 tls: {}
```

For each router, it defines websecure as the endpoint — requiring HTTPS for every router — along with HSTS as middleware. The same file defines the TLS configuration as follows:

```
tls:
 certificates:
 - certFile: /etc/traefik/certs/blume-gateway.crt
 keyFile: /etc/traefik/certs/blume-gateway.key
 options:
 default:
 minVersion: VersionTLS12
```

```
cipherSuites:
- TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
- TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
- TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
```

This includes the location of the TLS certificate (which contains the server's public key) and the corresponding private key used to establish encrypted connections. In the current configuration, applied locally, these are generated via an automatic key generation script at `infrastructure/security/generate-certs.sh`, though these certificates are not compatible across hosts.

The `docker-compose.yml` in `infrastructure` includes the following commands:

```
--entrypoints.web.http.redirects.entrypoint.to=websecure
--entrypoints.web.http.redirects.entrypoint.scheme=https
```

These instruct the system that when a connection is attempted on an HTTP port (port 80), it is immediately redirected to HTTPS before any application content is exchanged — subtly preventing the use of HTTP without directly blocking the user's request (provided the security policy is met, of course).

## Reverse Proxy

### Scenario

- **Source:** An external attacker or automated scanner attempting to map the internal architecture of the Blume platform.
- **Stimulus:** The attacker scans the host for open ports and attempts to communicate directly with internal microservice endpoints — for example, the Spring Boot business logic service on port 8082, the Go stream engine on port 8080, the Elixir/Phoenix activities service on port 4000, or the record service on port 8081. Alternatively, the attacker tries to access the Traefik administration dashboard to extract the internal routing table.
- **Artifact:** The *Traefik* API Gateway and the internal Docker network that exposes microservice ports exclusively within `blume_app`.
- **Environment:** The system is running under normal conditions. The host exposes only the following ports: 80 (HTTP, immediately redirected to HTTPS), 443 (HTTPS via Traefik), 1935 (RTMP for OBS streaming), and 8889 (WebRTC/WHEP for MediaMTX). All internal microservice ports are bound exclusively to Docker internal networks.

The attacker runs a port scan and attempts direct connections:

```
Attacker attempts to reach internal services directly
nc -zv localhost 8082 # Spring Boot – business logic
nc -zv localhost 8080 # Go – stream engine
nc -zv localhost 8081 # Go – record service
nc -zv localhost 8000 # Python – recommendations
nc -zv localhost 4000 # Elixir/Phoenix – activities
nc -zv localhost 8888 # MediaMTX HTTP API

Attacker attempts to extract the routing table from the Traefik dashboard
curl -s http://localhost:8080/api/rawdata
```

All attempts return `Connection refused` — no service is reachable on those ports from the host. The Traefik admin API is also disabled (`--api.insecure=false`), so the internal routing configuration cannot be extracted. The internal DNS names (`blume-business-logic-ms`, `blume_stream_ms`, etc.) are resolvable only inside the `blume_app` Docker network and are never communicated to any external client.

- **Response:** The reverse proxy (Traefik) is the exclusive external contact point. All client requests must enter through port 443. Traefik inspects each request's URL path, matches it against its routing table, and forwards it to



the correct internal service — without the client ever knowing the target service's address, port, or technology stack. The backend service responds to Traefik; Traefik relays the response back to the client.

A legitimate request successfully routes through the proxy:

```
Only Traefik's port is reachable – internal topology is invisible to the caller
curl -k -s -o /dev/null -w "%{http_code}" https://localhost/api/auth/login
→ 401 (routed internally to blume-business-logic-ms:8082, unknown to the caller)

curl -k -s -o /dev/null -w "%{http_code}" https://localhost/api/v1/recommendations
→ 200 (routed internally to blume_recommendations_ms:8000, unknown to the caller)
```

- **Response Measure:** (1) `nc -zv localhost <port>` returns `Connection refused` for all internal service ports (8082, 8080, 8081, 8000, 4000, 8888). (2) `curl http://localhost:8080/api/rawdata` returns `Connection refused` — the Traefik admin API is disabled. (3) HTTP responses are returned for valid paths through port 443 only. (4) No internal hostname, IP, or port number appears in any HTTP response header received by the client.

## SCENARIO 2: Reverse Proxy Pattern

```
[2.1] Internal microservice ports must NOT be reachable from host
PASS: Port 8082 (internal service) not exposed on host (got: closed)
PASS: Port 8080 (internal service) not exposed on host (got: closed)
PASS: Port 8081 (internal service) not exposed on host (got: closed)
PASS: Port 8000 (internal service) not exposed on host (got: closed)
PASS: Port 4000 (internal service) not exposed on host (got: closed)
PASS: Port 8888 (internal service) not exposed on host (got: closed)

[2.2] Traefik admin dashboard must be disabled
PASS: Traefik admin API inaccessible (:8080) (got: 000)

[2.3] Traffic must route successfully through Traefik (:443)
PASS: Requests route through Traefik (not connection refused) (got: 401)

[2.4] No internal address or port appears in response headers
PASS: Internal ports not leaked in response headers (got: clean)
```

### Characteristics

- **Threat:** Reconnaissance and direct exploitation of internal microservice endpoints.
- **Attack:** Port scanning and direct connection to backend service ports; extraction of the internal routing topology via the Traefik admin API; crafting raw HTTP requests aimed at bypassing gateway-level controls (authentication middleware, path validation) by contacting backend services directly.
- **Weakness:** Each microservice binds to its own port and handles incoming connections directly at the application layer, creating as many potential entry points as there are services. An attacker who discovers a backend port can communicate with that service as if they were the gateway.
- **Vulnerability:** If internal service ports were published to the host, an attacker could bypass Traefik entirely and send requests directly to Spring Boot (:8082), Go (:8080), or Phoenix (:4000) — potentially bypassing the JWT validation middleware and TLS enforcement that are applied at the application layer, not at the transport layer.
- **Risk:** Direct access to unauthenticated internal endpoints could allow unauthorized data reads, state mutation, or exploitation of service-specific vulnerabilities without passing through any centralized security boundary.

- **Countermeasure:** The Reverse Proxy pattern, implemented via Traefik, acts as a single choke point: all inbound HTTP/HTTPS traffic is funneled through it, internal service ports are never published to the host, and the routing configuration that reveals the internal topology is inaccessible from outside the container network.

## Architectural Tactics

### DETECT

- **Monitor:** Traefik's access log records every incoming request with its source IP, URL path, HTTP status, and response latency. This provides a centralized audit trail for all traffic entering the system — anomalous patterns (high 404 rates from a single IP, unusual path probing, scanning signatures) are visible in a single log stream without requiring each microservice to implement its own access logging.

### RESIST

- **Limit Exposure:** Internal microservice ports (8082, 8080, 8081, 8000, 4000, 8888) are declared only on the `blume_app` Docker network and are never published to the host via a `ports` mapping. The Traefik management API is disabled via `--api.insecure=false`, preventing an attacker from extracting the routing table even if they reach the gateway host.
- **Authenticate Actors:** Because Traefik is the sole entry point, it becomes the natural location to enforce cross-cutting security policies. TLS termination (Scenario 1) and the path-based routing rules that forward requests to JWT-protected services (Scenario 4) are all applied at this single point before any backend service is reached.
- **Limit Access:** The routing table in `traefik/dynamic.yml` explicitly declares which paths are routable. A request to an undeclared path will not match any router and will receive a 404 — no internal service is accidentally reachable via an undocumented URL.

### REACT

- **Inform:** HSTS headers (applied by the `hsts` middleware on every router) instruct browsers to never attempt plain HTTP connections, preventing downgrade attacks that could circumvent the HTTPS entry point.

### RECOVER

In the current implementation, the Reverse Proxy pattern does not define explicit recovery tactics beyond the `restart: unless-stopped` policy on the Traefik container, which ensures the proxy is automatically restarted if it crashes.

## Architectural Pattern

The **Reverse Proxy** pattern interposes a dedicated intermediary component between external clients and a set of internal services. From the client's perspective, there is exactly one server — the proxy. Internally, there is an arbitrary number of services at different addresses and ports. The proxy owns the mapping between public URL paths and internal backends; this mapping is never disclosed to clients.

In Blume, this pattern is implemented by Traefik. Its configuration in `infrastructure/traefik/dynamic.yml` defines a routing table that maps URL path prefixes to internal services by priority:

```
route:
 rule: "PathPrefix(`/api/v1`)"
 entryPoints: ["websecure"]
 middlewares: ["hsts"]
 priority: 15
 service: recommendations
 tls: {}
business:
 rule: "PathPrefix(`/api`)"
 entryPoints: ["websecure"]
 middlewares: ["hsts", "business-cb"]
 priority: 10
 service: business-proxy
 tls: {}
frontend:
 rule: "PathPrefix(`/`)"
 entryPoints: ["websecure"]
 middlewares: ["hsts"]
 priority: 1
```

```

 service: blume_wa
 tls: {}

services:
 recommendations:
 loadBalancer:
 servers:
 - url: "http://blume_recomendations_ms:8000"
 business-proxy:
 loadBalancer:
 servers:
 - url: "http://toxiproxy:38082"
 blume_wa:
 loadBalancer:
 servers:
 - url: "http://blume_wa:3000"

```

A client that sends `GET /api/v1/recommendations` to `https://blume-host` receives a response from the Python *FastAPI* service running at `blume_recomendations_ms:8000` — without ever knowing that address. Priority values resolve overlapping prefixes deterministically: `/api/v1/something` matches `recommendations` (priority 15) rather than `business` (priority 10). For the `/api` path, the routing chain involves an additional intermediary. *Traefik* forwards the request to `business-proxy`, which resolves to *ToxiProxy* at `:38082`. *ToxiProxy* then forwards it to `blume-business-logic-ms:8082` according to its own configuration in `blume_ag/toxiproxy/toxiproxy.json`:

```

[
 {
 "name": "business-logic-proxy",
 "listen": "0.0.0.0:38082",
 "upstream": "blume-business-logic-ms:8082",
 "enabled": true,
 "toxics": []
 }
]

```

From the client's perspective this chain is fully opaque — the response arrives from *Traefik*'s address regardless of how many internal hops were involved. The business router also applies the `business-cb` circuit-breaker middleware, which can short-circuit the chain at the *Traefik* level and return a synthetic 503 before the request reaches *ToxiProxy* or the backend at all.

The `docker-compose.yml` enforces the network invariant that makes this possible:

```

traefik:
 ports:
 - "80:80"
 - "443:443"
 networks:
 - blume_edge
 - blume_app

toxiproxy:
 networks:
 - blume_app

blume-business-logic-ms:
 - blume_app
 - blume_data

```



*Traefik* sits on both `blume_edge` (reachable from the host) and `blume_app` (reachable from services). Backend services sit only on `blume_app` with no `ports` entry — they are unreachable from the host by construction. The proxy is the only component that bridges both networks, making it the mandatory path for all external traffic.

## Network segmentation

---

An architectural security pattern that divides a network infrastructure into multiple isolated logical subnets, applying strict access policies between them to contain potential breaches and prevent unauthorized lateral movement. The primary advantage of this pattern is the significant reduction of the system's attack surface by enforcing the principle of least privilege at the network level. In Blume, this is implemented by segregating the container architecture into four distinct Docker networks: `blume_edge` (acting as a DMZ for perimeter traffic via *Traefik*), `blume_app` (a private network for application microservices), `blume_data` (an internal network configured with `internal: true`), and `postgres_tailscale` (a dedicated bridge network for the Elixir cluster). This ensures that infrastructure components like databases ( `MySQL` , `PostgreSQL` ), object storage ( `MinIO` ), and message brokers ( `RabbitMQ` ) have no host-level port exposure and can exclusively receive traffic from authorized microservices attached to their segment.

## Scenario

- **Source:** Bad external actor or compromised process in the edge zone (`blume_edge`: *Traefik*, frontend), that tries to reach resources in internal layers without authorization. Includes: Attacker in the host's network, container in DMZ after exploitation, or automatic port scanning.
- **Stimulus:** Direct network access attempt to data and message services (`MySQL` :3306, `PostgreSQL` :5432, `RabbitMQ` :5672, `MinIO` :9000) without going through the HTTP gateway (*Traefik*) or the authorized app logic. Examples: `nc mysql 3306`, scanning of ports in the host, JDBC/AMQP connection from a container only on `blume_edge`.
- **Artifact:** The `blume_data` Docker network and the data-layer services attached to it (`MySQL` primary & warm-spares, `PostgreSQL`, `RabbitMQ`, `MinIO`).
- **Environment:** Local deployment with Docker Compose in a node (four networks: `blume_edge`, `blume_app`, `blume_data` with `internal:true`, and `postgres_tailscale`). Conceptual parity with production in AWS (public/private subnets, Security Groups ALB → ECS → RDS). The system is in normal operation (deployed stack, users can use HTTP in :80).
- **Response:** The system denies unauthorized TCP connections between zones: the DMZ and the application layer without data attachment don't establish session with DB, queue or storage. The legitimate traffic of users keeps entering by *Traefik* (:80); the microservices that require persistence access `blume_data` only if they are in `blume_app` and `blume_data`.
- **Response measure:** (1) 0 successful connections from probe in `blume_edge` towards `mysql`, `recording-mysql`, `postgres`, `rabbitmq` and `minio` (automated test). (2) 0 connections from probe only in `blume_app` towards `mysql` and `rabbitmq`. (3) Connection permitted only from probe with networks `blume_app` + `blume_data` (simulates legitimate microservice). (4) Edge availability: HTTP 200 in public routes via *Traefik* (i.e. `/`, `/api/v1/health`). (5) Surface on host: ports of not published data (3306, 5672, 9000 not mapped to host). Acceptance criteria: `run-test.sh` script ends with exit code 0 and 12/12 PASS comprobations.

```

=== Prueba de segmentación de red – Blume (Seguridad) ===
Directorio: /home/turing/Documents/UNAL/1C/infrastructure

--- Escenario 1: zona DMZ (blume_edge) no alcanza capa de datos ---
[PASS] DMZ → MySQL (business) bloqueado
[PASS] DMZ → MySQL (recordings) bloqueado
[PASS] DMZ → RabbitMQ bloqueado
[PASS] DMZ → PostgreSQL bloqueado
[PASS] DMZ → MinIO bloqueado

--- Escenario 2: capa de aplicación sin datos no alcanza BD/cola ---
[PASS] App (sin data) → MySQL bloqueado
[PASS] App (sin data) → RabbitMQ bloqueado

--- Escenario 3: servicios con rol app+data sí alcanzan datos ---
[PASS] App+Data → MySQL permitido (simula microservicio con acceso legítimo)
[PASS] App+Data → RabbitMQ permitido

--- Escenario 4: punto de entrada HTTP solo en DMZ ---
[PASS] DMZ → Traefik:80 permitido (único HTTP público del stack)
[PASS] DMZ → business-logic directo bloqueado (solo vía Traefik)

--- Escenario 5: gateway enruta a aplicación (control funcional) ---
[PASS] Traefik enruta API de recomendaciones

=== Resumen: 12 pass, 0 fail ===

```

## Characteristics

- **Threat:** Unauthorized access to sensitive data and lateral movement from a lower-trust component (frontend/DMZ) toward critical infrastructure (credentials in DB, messages in RabbitMQ, recordings in MinIO). Objectives: theft of user/channel data, manipulation of recording queues, denial of service on persistence layers.
- **Attack:** (1) Scanning and direct connection to DB/queue/storage ports exposed on the host or within a shared flat network. (2) Pivoting from a compromised edge container toward MySQL/PostgreSQL using default or leaked credentials. (3) Bypassing the API Gateway by calling microservices or databases via internal IP/port without application-layer authentication.
- **Weakness:** The deployment architecture has a single network plane ( `blume_net` ): all containers share the same logical Docker L2/L3 segment, placing frontend, application, and data-layer services as peers on the same network. This allows any compromise in a service to reach any other open port via DNS name ( `mysql` , `rabbitmq` ).
- **Vulnerability:** Exposure of infrastructure ports to the host (3306, 5672, 15672, 9000, 9001) and lack of isolation between Docker networks: a process on the host or in an edge container can open a TCP session against MySQL/RabbitMQ/MinIO without passing through application-layer controls (JWT, business authorization).
- **Risk:** High in the unsegmented state (medium-high probability × high impact): direct exposure of personal and business data, potential data exfiltration or DB deletion, abuse of recording queues. Medium-low after the countermeasure: direct attack from DMZ/app without data access is blocked; residual risks persist (ports :4000, RTMP/WebRTC, `/internal/*` routes via Traefik, weak credentials in RabbitMQ).
- **Countermeasure:** Tactic: network segmentation. Implementation: (1) three Docker networks ( `blume_edge` , `blume_app` , internal `blume_data` ); (2) minimal network attachment per service (only microservices requiring it join `blume_data` ); (3) removal of `ports` mapping in data services; (4) Traefik :80 as the single public HTTP endpoint; (5) MinIO accessed via `/storage` in the gateway; (6) on AWS: VPC, public/private subnets, Security Groups (ALB → ECS → RDS). Verification: `tests/security/network-segmentation/run-test.sh`
- Network Segmentation Table:

| Service  | blume_e<br>dge | blume_<br>app | blume_data |
|----------|----------------|---------------|------------|
| traefik  | ✓              | ✓             |            |
| blume_wa | ✓              |               |            |

|                                                         |  |   |                                                       |
|---------------------------------------------------------|--|---|-------------------------------------------------------|
| Microservices (Spring, Go, FastAPI, Phoenix)            |  | ✓ | ✓ if using DB/queue/storage                           |
| blume_recommendations_ms                                |  | ✓ |                                                       |
| mysql, mysql-warm, postgres, rabbitmq, recordings-mysql |  |   | ✓                                                     |
| minio                                                   |  | ✓ | ✓ (S3 API only from app; public via Traefik /storage) |
| mediamtx                                                |  | ✓ |                                                       |

Network segmentation is primarily a **Resist / Prevent** tactic: *isolation* between zones, *communication path control*, and *least privilege* connectivity to contain lateral movement before damage occurs.

## Architectural Tactics

### DETECT

- **Monitor:** Traefik's access log records every incoming request with its source IP, URL path, HTTP status, and response latency. Any attempt to reach an undeclared path returns a 404 that is visible in the centralized log stream, exposing port-scanning and probing patterns without requiring each microservice to implement its own access logging.

### RESIST

- **Separate Entities:** Three distinct Docker networks are defined — `blume_edge` (public-facing DMZ), `blume_app` (application tier), and `blume_data` (persistence tier, declared with `internal: true`). Docker enforces physical isolation between them: a container on `blume_edge` cannot open a socket to any host on `blume_data` because there is no routing path between the two networks.
- **Limit Exposure:** No data-layer service ( `mysql` , `mysql-warm` , `recordings-mysql` , `postgres` , `rabbitmq` , `minio` ) publishes a `ports` mapping to the host. Their ports are accessible only through the `blume_data` internal network, which has no host-level or edge-level interface.
- **Limit Access:** The routing table in `traefik/dynamic.yml` declares only the paths that are intentionally routable. Requests to undeclared paths receive a 404 — no internal service is reachable through an undocumented URL.
- **Authenticate Actors:** Only services explicitly attached to both `blume_app` and `blume_data` can resolve the data-service Docker DNS names ( `mysql` , `rabbitmq` , etc.). A container present only on `blume_edge` or `blume_app` without `blume_data` attachment fails DNS resolution before any TCP connection is attempted.

### REACT

- **Inform:** HSTS headers on every Traefik router prevent browsers from downgrading connections to plain HTTP, eliminating a class of interception vectors that could otherwise be used to extract internal topology details from response headers.

### RECOVER

In the current implementation, the Network Segmentation pattern does not define explicit recovery tactics. The `restart: unless-stopped` policy on all containers ensures automatic restart after a crash, but no point-in-time restore or audit trail mechanism has been implemented.

---

## Architectural Pattern

The **Network Segmentation** pattern divides the infrastructure into isolated logical zones with explicitly controlled trust boundaries between them. Access between zones is not merely restricted — it is structurally impossible at the network level for non-authorized paths.

In Blume, this is implemented through three Docker networks in `docker-compose.yml` :

```
networks:
 blume_edge:
 blume_app:
 blume_data:
 internal: true
```

The `internal: true` flag on `blume_data` instructs Docker to create a network with no external routing — not even to the host. Services are attached to the minimum set of networks their function requires:

| Service                                                 | blume_edge | blume_app | blume_data             |
|---------------------------------------------------------|------------|-----------|------------------------|
| traefik                                                 | ✓          | ✓         |                        |
| blume_wa                                                | ✓          |           |                        |
| Microservices (Spring, Go, FastAPI, Phoenix)            |            | ✓         | ✓ (if DB/queue needed) |
| blume_recommendations_ms                                |            | ✓         |                        |
| mysql, mysql-warm, postgres, rabbitmq, recordings-mysql |            |           | ✓                      |
| minio                                                   |            | ✓         | ✓                      |
| mediamtx                                                |            | ✓         |                        |

A container on `blume_edge` cannot initiate a connection to `blume_data` because Docker provides no route between them. Traffic must flow Edge → App → Data, and only microservices that legitimately require database or queue access are placed on `blume_data`. This enforces least-privilege connectivity at the infrastructure level, independent of any application-layer authentication.

## Token-based Authentication Pattern

### Scenario

- **Source:** A malicious student or an external attacker.
- **Stimulus:** They attempt to start a live stream on MediaMTX (an action reserved for professors), or try to connect to the real-time chat by impersonating another user.
- **Artifact:** The service boundaries of `blume_stream_ms` (Go), `blume_business_logic_ms` (Spring Boot), and `blume_stream_activities_ms` (Elixir/Phoenix), where JWT verification is enforced independently.
- **Environment:** The system is operating under normal conditions. Without token validation at each service boundary, any request that reaches `blume_stream_ms` or `blume_stream_activities_ms` would be processed regardless of the caller's identity or role.

The attacker attempts three different unauthorized actions:

```
Attack 1 – Attempting to publish an RTMP stream without credentials
curl -k -s -X POST https://localhost/auth/mediamtx \
 -H "Content-Type: application/json" \
 -d '{"action":"publish","path":"/live/stream-key","query":"","ip":"1.2.3.4"}'

Attack 2 – Forging a JWT to claim the PROFESSOR role (wrong secret)
FAKE_TOKEN="eyJhbGciOiJIUzI1NiJ9.eyJyb2xlQ29kZSI6I1BST0ZFUiNPUIJ9.INVALIDSIG"
curl -k -s -H "Cookie: blume_session=$FAKE_TOKEN" https://localhost/api/channels

Attack 3 – Connecting to the real-time chat WebSocket without a token
wscat --no-check -c "wss://localhost/socket/websocket"
```

In all three cases, the request reaches the respective service (Go, Spring Boot, Elixir/Phoenix) where JWT verification is enforced independently.

- **Response:** Each service verifies the JWT signature cryptographically before executing any business logic. If the token is absent, forged, expired, or carries an insufficient role, the request is immediately rejected — an HTTP 401/403 is returned or the WebSocket handshake is closed — without consulting any external service or shared session store.

```
Attack 1 – No token: rejected by Go mediamtx_handler.go
→ 401 Unauthorized: "unauthorized: missing token"

Attack 2 – Invalid signature: rejected by Spring Boot JwtAuthFilter.java
→ 403 Forbidden (user treated as anonymous, access denied by Spring Security)

Attack 3 – No token: rejected by Elixir/Phoenix UserSocket.connect/3
→ Connection closed – :error returned before any resource is allocated
```

An additional integrity check demonstrates that even a valid token with a modified payload is rejected:

```
Tampered payload: student changes roleCode to PROFESSOR in the JWT body
Any change to header or payload invalidates the HMAC signature
TAMPERED="<base64url(header)>.<base64url(payload_with_roleCode_PROFESSOR)>.<original_sig>"
curl -k -s -o /dev/null -w "%{http_code}" \
 -H "Cookie: blume_session=$TAMPERED" https://localhost/api/channels
→ 403 Forbidden – signature mismatch detected
```

- **Response Measure:** (1) Requests without a token to `/auth/mediamtx` return 401. (2) Requests with a forged or tampered JWT to `/api/*` return 403. (3) WebSocket connection attempts without a `token` parameter are rejected before any channel is joined. (4) Student tokens attempting the `publish` action return 403 from `mediamtx_handler.go`. (5) All checks pass in under 100ms, with no impact on legitimate authenticated users. Full automated verification: `tests/security/jwt-reverse-proxy/run-test.sh`.

## SCENARIO 4: JWT Token-based Authentication Pattern

```
[4.1] REST layer – request without token must be rejected
PASS: No token → 403 Forbidden (got: 403)

[4.2] REST layer – forged token (invalid signature) must be rejected
PASS: Forged token (wrong signature) → 403 Forbidden (got: 403)

[4.3] Streaming layer (Go) – publish without token must be rejected
PASS: Publish without token → 401 Unauthorized (got: 401)

[4.4] Streaming layer (Go) – student token publishing must be forbidden
PASS: Student publish attempt (invalid token) → 401 Unauthorized (got: 401)

[4.5] REST layer – tampered payload must be rejected (integrity check)
PASS: Tampered JWT payload → 403 Forbidden (got: 403)
```

### Characteristics

- **Threat:** Unauthorized access to professor-exclusive features (live streaming); identity spoofing in the real-time chat.
- **Attack:** (1) Attempting to publish an RTMP stream without credentials; (2) connecting to a WebSocket channel without a token; (3) forging a JWT with a self-assigned `PROFESSOR` role using a wrong secret; (4) tampering with the payload of a legitimately obtained token to escalate privileges.

- **Weakness:** Each service has to go through one central authentication point to verify identity, thus becoming one single point of failure that could then be compromised (both from the security and the reliability perspective), the alternative of blindly trusting each request would also be considered a weakness because it doesn't provide identity guarantees.
- **Vulnerability:** An attacker who bypasses Traefik (e.g. by exploiting the network weakness described in Scenario 3) could reach services that trust all incoming connections, gaining unauthorized access to streaming and chat resources.
- **Risk:** A student posing as a professor could hijack the RTMP publish slot and disrupt a live class session; an attacker could flood the real-time chat under any fabricated identity, compromising the integrity of live interactions.
- **Countermeasure:** Stateless JWT-based authentication applied independently at all three service boundaries (Spring Boot, Go, Elixir/Phoenix), sharing only the cryptographic secret ( `JWT_SECRET` ). No shared session store is required; each service can verify identity and role autonomously.

## Architectural Tactics

### DETECT

- **Verify Message Integrity:** The HMAC signature of the JWT acts as an integrity mechanism. Any alteration to the header or payload — even a single byte — produces a completely different signature, which the receiving service detects and rejects. This allows each service to independently confirm that the token has not been tampered with since it was issued by the trusted authority (Spring Boot).

### RESIST

- **Authenticate Actors:** Each service cryptographically validates the JWT signature using the shared `JWT_SECRET` with the HMAC algorithm (HS256/HS384). A token signed with any other key — or one whose payload has been altered — is rejected before any business logic executes.
- **Authorize Actors:** Beyond authentication, `mediamtx_handler.go` enforces role-based access control: the `roleCode` claim must equal `PROFESSOR` for `publish` actions. Role information is embedded in the token itself, so no database query is needed to make the authorization decision.
- **Identify Actors:** `StreamActivitiesWeb.UserSocket` extracts `userId` and `username` from the verified token and binds them to the WebSocket connection state (Socket Assigns). Every chat message sent over that connection is cryptographically tied to the validated sender identity.
- **Limit Exposure:** Token expiration ( `JWT_EXPIRATION_SECONDS` ) serves as the revocation mechanism—a compromised token becomes invalid once it expires, reducing the attack window without requiring server-side state.

### REACT

In the current implementation, the Token-based Authentication pattern does not define explicit reaction tactics. If a token is compromised before it expires, no active revocation mechanism is in place; the attack window is bounded by `JWT_EXPIRATION_SECONDS`.

### RECOVER

In the current implementation, the Token-based Authentication pattern does not handle explicit recovery mechanisms.

## Architectural Pattern

The **Token-based Authentication** pattern (also known as **Bearer Token** or **Stateless Auth**) delegates authentication state from the server to the client. Instead of maintaining server-side sessions — which would require a shared session store across microservices — the server issues a cryptographically signed token at login time. The client attaches that token to every subsequent request; each receiving service verifies the signature independently, without consulting any database or central authority.

This pattern is structurally essential in a microservices architecture: since Spring Boot, Go, and Elixir/Phoenix are completely independent processes, sharing session state between them would require additional infrastructure (Redis, a shared database). The JWT eliminates that need — the shared secret ( `JWT_SECRET` ) is the only dependency among the three verifiers.

In Blume, the token is generated in `JwtTokenAdapter.java` at login time:

```
// JwtTokenAdapter.java – token generation in Spring Boot
String token = Jwts.builder()
 .subject(userInfo.email())
 .claim("userId", userInfo.userId().toString())
 .claim("username", userInfo.username())
 .claim("roleCode", userInfo.roleCode()) // authorization claim read by Go and Elixir
 .issuedAt(now)
 .expiration(expiry)
 .signWith(key) // HMAC-SHA, key derived from JWT_SECRET
 .compact();
```

The token travels as an `httpOnly` cookie ( `blume_session` ). Each microservice verifies it autonomously:

- **Spring Boot — `JwtAuthFilter.java`** : Intercepts every HTTP request, extracts the token from the cookie, calls `validateSession.validate(token)` (which delegates to `JwtTokenAdapter.parse()` ), and populates the `SecurityContext` with the authority `ROLE_<roleCode>` . If the token is invalid or absent, the user remains anonymous and Spring Security enforces the path-level access restrictions.
- **Go — `mediamtx_handler.go`** : Extracts the token from the `?token=` query param that MediaMTX forwards in the webhook payload. It verifies it with `jwt.Parse()` , accepting any HMAC variant (HS256, HS384, HS512), and applies the hard business rule: only `roleCode == "PROFESSOR"` may execute the `publish` action. For `read` actions, any authenticated user is accepted.
- **Elixir/Phoenix — `StreamActivitiesWeb.UserSocket`** : The `connect/3` callback verifies the token using the `Joken` library, trying HS256 first and then HS384 for compatibility with the algorithm `jjwt` selects based on key length. If verification fails, it returns `:error` and the WebSocket connection is never established — the attacker consumes no chat server resources.

All three verifiers are **stateless** and **independent**: they can scale horizontally without coordination. The pattern is implemented as follows in `docker-compose.yml` , where the single shared secret is injected as an environment variable into every service that needs to verify tokens:

```
blume-business-logic-ms:
 environment:
 JWT_SECRET: ${JWT_SECRET} # issues tokens

blume_stream_ms:
 environment:
 JWT_SECRET: ${JWT_SECRET} # verifies tokens (Go)

blume_stream_activities_ms:
 environment:
 JWT_SECRET: ${JWT_SECRET} # verifies tokens (Elixir)
```

No service needs to call another to validate a token — the shared secret is the entire trust contract.

## Performance and Scalability

### Load balancing

#### Scenario

- **Source**: Virtual users simulated by k6, representing concurrent students browsing the public course catalog.
- **Stimulus**: 700 concurrent requests arrive at `GET /api/cursos/explorar`, a load that exhausts the HikariCP connection pool of a single `blume_business_logic_ms` replica and produces connection acquisition timeouts.
- **Artifact**: Traefik's weighted round-robin load balancer for the business service, distributing incoming requests across the registered replicas of `blume_business_logic_ms`.



- **Environment:** Normal operation under peak load; two replicas of `blume_business_logic_ms` are active and registered behind Traefik; each replica holds an independent HikariCP connection pool of 10 connections to MySQL.
- **Response:** Traefik distributes incoming requests across both replicas using weighted round-robin; each replica receives approximately half the concurrent load (~350 requests), keeping its individual HikariCP pool below exhaustion. No replica accumulates a connection acquisition queue large enough to trigger the 30-second timeout, and no request fails due to pool saturation.
- **Response Measure:** `http_req_failed` remains at 0% at 700 VUs — a load level that produced ~0.50% failures on a single replica in Lab 6; the effective knee of the curve shifts from ~575 VUs (1 replica) to ~1,150 VUs (2 replicas, each with an independent pool of 10 connections); no HikariCP acquisition timeout is exceeded on any individual instance.

## Applied architectural tactics

- **Introduce Concurrency / Maintain Multiple Copies of Computation:** `blume_business_logic_ms` runs with `deploy.replicas: 3` in `infrastructure/docker-compose.yml`, each replica as an independent JVM process with its own HikariCP connection pool. The scenario evaluates a two-replica configuration (reachable via `docker compose up --scale blume-business-logic-ms=2`, as documented in the compose file itself), where each replica absorbs approximately half of the 700 concurrent VUs (~350), keeping individual pool pressure below the saturation threshold.
- **Round-Robin Request Distribution:** The `business` router in `traefik/dynamic.yml` points to `service: business@docker`. Traefik's Docker provider discovers each replica's individual IP and applies WRR with equal implicit weights (`weight = 1`), which is functionally equivalent to plain round-robin: each incoming request is forwarded to the next replica in sequence. No differentiated WRR weights are active in the current configuration.
- **Manage Resources / Bound Queue Size:** The HikariCP pool (10 connections per replica) acts as a hard gate on concurrent MySQL access. On a single replica at 700 VUs, the acquisition queue grows until the 30-second timeout fires. With two replicas each receiving ~350 VUs, neither pool accumulates a queue large enough to reach that threshold.
- **Health-Check-Based Exclusion:** The service labels in `docker-compose.yml` configure an active health check against `/api/health` (interval: 10 s, timeout: 3 s). This mechanism operates as **exclusion**: if a replica stops responding to probes, Traefik removes it from the active rotation. It does not guarantee that a newly started replica is validated before receiving traffic — its function is to remove degraded instances from the pool, not to gate admission.
- **Horizontal Scale-Out:** Capacity is increased by raising `deploy.replicas` without modifying any existing replica or any balancer configuration. The effect is linear: each additional replica contributes 10 independent HikariCP connections, shifting the saturation knee proportionally.

## Applied architectural patterns

**Load Balancing via Reverse Proxy (Round-Robin with Docker Provider):** Traefik acts as a reverse proxy on the `blume_app` network and manages the replica pool of `blume_business_logic_ms` directly through the Docker provider. When the Docker daemon registers a new replica container, Traefik reads its dynamically assigned IP and incorporates it into the routing table under `business@docker`. The caller — `blume_wa` or `blume_ma` — sends all requests to a single stable address; replica selection happens entirely in the infrastructure layer, invisible to the client.

**Shared-Nothing Horizontal Replicas:** Each replica of `blume_business_logic_ms` is a fully independent process with no shared mutable state between instances. Specifically, each holds its own HikariCP connection pool and its own JVM heap, both bounded by `deploy.resources.limits` (1 CPU, 600 MB RAM per replica as defined in the compose file). Pool saturation or a GC pause on one replica has no effect on the others. This design allows total system capacity to scale by adding replicas with no inter-instance coordination.

## Caching

### Scenario

- **Source:** Students accessing the recommendation feed through `blume_wa` or `blume_ma`.



- **Stimulus:** 10 concurrent requests for GET /api/v1/recommendations arrive within the 2-minute TTL window.
- **Artifact:** The in-memory Cache-Aside store in `blume_recommendations_ms` (`app/core/cache.py`), which holds the aggregated response built from `blume_business_logic_ms` and `blume_stream_activities_ms`.
- **Environment:** Normal operation; the cache has been warmed by at least one prior successful request within the last 2 minutes; the stored entry is valid and has not expired.
- **Response:** `blume_recommendations_ms` checks the cache via `streams_repo.py`, finds a valid entry, and returns the recommendations response directly from memory — without issuing any HTTP call to `blume_business_logic_ms` or `blume_stream_activities_ms`.
- **Response Measure:** 10 concurrent requests within the TTL window generate 0 upstream calls instead of 20; p99 response time on cache-hit requests is at least 60% lower than on cache-miss requests.

### Applied architectural tactics

- **Cache (Maintain Multiple Copies of Data):** `recommendation_service.py` stores the complete, already-scored `RecommendationResult` in a `TTLCache` instance (defined in `app/core/cache.py`) keyed by `recommendations:{user_id or ''}:{limit}` with a TTL of 120 seconds. Subsequent requests with the same parameters within that window read directly from memory and return without executing any further operation.
- **Reduce Computational Overhead:** On a cache miss the service executes a full pipeline: two parallel HTTP calls via `asyncio.gather` (one to `blume_business_logic_ms`, one to `blume_stream_activities_ms`), response deserialization and schema validation, and the scoring algorithm (floating-point arithmetic plus a sort over the full stream list). On a cache hit the function returns at the very first code block, before any of those steps is initiated — this is the structural reason the p99 latency drops by at least 60%.
- **Limit Event Response via TTL:** The 120-second TTL on the response-level cache is aligned with the TTLs of the underlying repositories (`STREAMS_CACHE_TTL = 120`, `ENGAGEMENT_CACHE_TTL = 120`, both defined in `config.py`). All layers expire within the same window, preventing the response cache from serving an aggregation built on data that its source caches would already consider stale.
- **Reduce Coupling to Upstream Services:** While a cache entry is valid, `blume_recommendations_ms` issues no HTTP calls to `blume_business_logic_ms` or `blume_stream_activities_ms`. A transient degradation in either upstream during the TTL window has zero impact on recommendation availability.

### Applied architectural patterns

**Cache-Aside (Lazy Population):** All cache logic lives in `recommendation_service.py` — not in a middleware layer nor in `streams_repo.py`. The pattern follows three explicit steps in the code: (1) check the `TTLCache` for the key (`user_id`, `limit`) before doing any work; (2) on a miss, execute the full pipeline and store the resulting `RecommendationResult` via `cache.set()`; (3) return the result, whether from memory or freshly computed. The cache populates lazily — only when a key is actually requested — so no memory is consumed for parameter combinations that have never been queried.

**Layered Caching:** The system implements two independent cache levels built on the same `TTLCache` class from `app/core/cache.py`. The upper level sits in `recommendation_service.py` and stores the complete aggregated response (streams + engagement + scoring). The lower level sits in the repositories: `streams_repo.py` caches the stream list (`all_streams`, TTL 120 s) and `activities_repo.py` caches engagement data (`engagement`, TTL 120 s) and per-user activity (`user:{user_id}`, TTL 300 s). On an upper-level cache hit, neither repository cache is consulted. On an upper-level miss but a repository hit, HTTP calls are avoided but the scoring algorithm still runs. This layering is viable without fragmentation concerns because `blume_recommendations_ms` is deployed as a single instance in the compose file (no `deploy.replicas`), meaning the in-memory cache is never split across processes.

## Performance testing analysis and results

### Load Balancing — k6 ( `scaling_test.js` )

**Tool:** k6 v2.0.0

**Endpoint:** GET /api/cursos/explorar

**Load profile:**

| Stage                | Duration | Target VUs |
|----------------------|----------|------------|
| Warm-up              | 20 s     | 50         |
| Low load             | 30 s     | 200        |
| Single-replica knee  | 30 s     | 575        |
| Hold (scenario load) | 60 s     | 700        |
| Ramp-down            | 20 s     | 0          |

**Threshold:** http\_req\_failed rate == 0 (zero-failure requirement)

**Run 1 — 1 active replica (baseline)**

```
http_req_failed: x rate=11.13% (4 001 failures / 35 922 requests)
status is 200: x 88.86%

http_req_duration:
 avg 714.61 ms
 med 587.29 ms
 p(90) 1.57 s
 p(95) 1.93 s
 max 11.69 s

http_reqs: 35 922 (223.39 req/s)
```

**Threshold result: FAILED** — HikariCP pool (10 connections) exhausted under 700 VUs; connection acquisition timeouts began before the 700-VU stage was fully reached. 11.13% of all requests returned non-200 responses.

**Run 2 — 2 active replicas (scenario configuration)**

```
http_req_failed: ✓ rate=0.00% (0 failures / 37 028 requests)
status is 200: ✓ 100.00%

http_req_duration:
 avg 661.74 ms
 med 23.74 ms
 p(90) 2.28 s
 p(95) 3.00 s
 max 8.74 s

http_reqs: 37 028 (230.23 req/s)
```

**Threshold result: PASSED** — Round-robin distribution across 2 replicas kept each replica's effective concurrent load at ~350 VUs, below the HikariCP pool saturation threshold. Zero failures recorded across the full 700-VU hold stage.

**Comparative summary**

| Metric             | 1 replica       | 2 replicas      | Change       |
|--------------------|-----------------|-----------------|--------------|
| http_req_failed    | 11.13%          | 0.00%           | −11.13 pp    |
| Requests succeeded | 31 921 / 35 922 | 37 028 / 37 028 | 100% success |

|                      |              |              |                                                  |
|----------------------|--------------|--------------|--------------------------------------------------|
| Median response time | 587.29 ms    | 23.74 ms     | −95.9%                                           |
| p(90)                | 1.57 s       | 2.28 s       | +0.71 s (tail from pool queueing under high VUs) |
| Max response time    | 11.69 s      | 8.74 s       | −25.2%                                           |
| Throughput           | 223.39 req/s | 230.23 req/s | +3.1%                                            |
| Threshold            | ✗ FAILED     | ✓ PASSED     |                                                  |

**Note on p(90) increase:** With 2 replicas the test completed more total requests (37 028 vs 35 922) because no iterations were aborted by timeouts. The higher p(90) in Run 2 reflects real queueing at the upper VU range — not degradation — since 0 requests failed.

Caching — pytest + precision timing ( test\_recommendations\_cache.py )

**Tool:** pytest 8.3.5 + Python time.perf\_counter()  
**Setup:** upstream repositories patched with AsyncMock applying 50 ms simulated latency per call, matching the order of magnitude of real inter-service HTTP latency. Cache reset between test groups via the autouse fixture ( recommendation\_service.\_cache = None ).

Upstream call count — correctness tests

| Test                                           | Result   | Assertion                                                     |
|------------------------------------------------|----------|---------------------------------------------------------------|
| test_cache_miss_calls_repos_once               | ✓ PASSED | get_all_streams called 1×;<br>get_stream_engagement called 1× |
| test_cache_hit_skips_repos                     | ✓ PASSED | Second identical call: both repos called 0 additional times   |
| test_n_concurrent_requests_zero_upstream_calls | ✓ PASSED | 10 concurrent requests on warm cache → repos called 0× total  |

N=10 concurrent requests within TTL window → **0 upstream calls** (vs 2N=20 without cache).

Latency measurement — 10 runs each

**Cache Miss** (50 ms simulated latency × 2 repos in parallel):

| Metric | Value    |
|--------|----------|
| avg    | 57.45 ms |
| p50    | 54.92 ms |
| p99    | 66.98 ms |
| min    | 52.31 ms |
| max    | 69.98 ms |

**Cache Hit** (warm cache, in-memory dict lookup + threading.Lock ):

| Metric | Value |
|--------|-------|
|--------|-------|

|     |           |
|-----|-----------|
| avg | 0.0064 ms |
| p50 | 0.0039 ms |
| p99 | 0.0149 ms |
| min | 0.0034 ms |
| max | 0.0180 ms |

Latency reduction summary

| Metric | Miss     | Hit       | Reduction |
|--------|----------|-----------|-----------|
| avg    | 57.45 ms | 0.0064 ms | 99.99%    |
| p99    | 66.98 ms | 0.0149 ms | 99.98%    |

p99 hit latency is **99.98% lower** than p99 miss latency — exceeding the  $\geq 60\%$  requirement by a wide margin. The reduction holds regardless of upstream latency magnitude: on a cache hit, zero time is spent waiting for HTTP responses from `blume_business_logic_ms` or `blume_stream_activities_ms`.

Concurrent hit throughput

N = 10 concurrent requests, cache warm  
Total wall time: 0.395 ms  
Avg per request: 0.040 ms  
Upstream calls: 0 (avoided 2N = 20 calls)

All 4 cache tests passed: 4 passed in 3.01 s

Visualización de resultados — Load Balancing

The following charts consolidate the results from the fixed-point tests and the official scaling script, comparing the behavior of 1 replica versus 2 replicas across different levels of concurrent load.

## Análisis de rendimiento: Load Balancing con Traefik blume\_business\_logic\_ms — GET /api/cursos/explorar



**Methodological note:** Fixed-point tests start directly at each VU level without a prior ramp-up, generating an initial connection spike that does not occur under normal scaling conditions. The result that validates the architectural scenario is the **official script with gradual ramp-up** (marked with ★ in each chart), where 2 replicas sustained 700 VUs with **0.00% failures**.

### Failure rate vs concurrent VUs

With 1 replica, the failure rate scales consistently from 300 VUs, reaching 5.31% at 700 VUs. With 2 replicas under fixed-point tests, behavior is similar at lower ranges; however, the critical difference appears in the official script: under gradual ramp-up, Traefik distributed load such that no replica accumulated a HikariCP acquisition queue large enough to trigger the 30-second timeout, and the failure rate held at **0.00%** while sustaining 700 VUs.

## p95 latency vs concurrent VUs

Both configurations show increasing latency as load grows. The 1,150-VU point with 2 replicas exhibits an apparent p95 drop that should not be interpreted as a performance improvement: with 44.24% failures at that level, failed requests complete faster than successful ones, biasing the latency distribution downward. Under the official script, p95 landed at 512 ms — controlled and near the 500 ms reference threshold — consistent with the even distribution of ~350 effective VUs per replica.

## Throughput (RPS) vs concurrent VUs

Throughput scales similarly for both configurations up to 575 VUs. Beyond that point, 1 replica begins saturating its HikariCP pool and the curve flattens with growing failures. With 2 replicas, throughput extends to ~634 RPS at 900 VUs before degrading. The official script recorded 340 RPS at 700 VUs — lower than the fixed-point peaks, which is expected: gradual ramp-up does not produce the same instantaneous burst, but yields a stable system with zero errors throughout the hold stage.

## Architectural validation

The official scaling script with gradual ramp-up is the test that validates the scenario: 2 replicas behind Traefik processed 54,705 requests with **0.00% failures** while sustaining 700 concurrent VUs for 60 seconds. This confirms that Traefik's round-robin distribution keeps the effective load on each replica below the HikariCP pool saturation threshold (10 connections), shifting the knee of the curve from ~575 VUs (1 replica) to at least 700 VUs sustained with zero failure tolerance.

## Reliability

### Hot Replication

#### Scenario

Failure of an active instance of the business logic microservice during normal operation.

#### SOURCE

System operator / spontaneous Docker container failure.

#### STIMULUS

Abrupt shutdown of 1 out of 3 replicas of blume-business-logic-ms while concurrent requests arrive at GET /api/cursos/explorar.

#### ARTIFACT

Active set of blume\_business\_logic\_ms replicas (Active/Active pattern) behind Traefik; Docker's internal DNS distributes traffic across instances (deploy.replicas: 3).

#### ENVIRONMENT

Normal operation; Docker Compose stack on a single host; public endpoint via <https://localhost>.

#### RESPONSE

Traefik automatically routes to surviving replicas; the service continues responding without manual intervention.

#### RESPONSE MEASURE

(1)  $\geq 3$  active replicas before the failure. (2) http\_req\_failed = 0 across 15 consecutive requests (1/s) after the failure. (3) Recovery latency  $\approx 0$  s (transparent failover to the client). RTO  $\approx 0$  s, RPO = 0 (stateless replicas; state stored in MySQL).

```

● UNAL/1C/infrastructure main !? > bash tests/replication/hot/run-test.sh

=====
REPLICATION HOT – Active/Active (blume_business_logic_ms)
=====

PASS: Al menos 3 réplicas activas de business-logic (got: 3)
PASS: Endpoint público responde antes del fallo (HTTP 200)

Simulando fallo de réplica: docker stop 0da1d98bdb92...
PASS: Peticiones fallidas durante 15 s post-fallo (objetivo: 0) (got: 0)

=====

HOT REPLICATION
Results: 4/4 PASS | 0 FAIL
=====

```

## Applied architectural tactics

- **Maintain Multiple Copies of Computation (Active Redundancy):** `docker-compose.yml` declares `deploy.replicas: 3` for `blume-business-logic-ms`, each replica running as an independent JVM process with its own HikariCP pool. All three replicas serve application traffic simultaneously (Active/Active). Because every request carries a self-contained JWT and all persistent state lives in the shared `mysql` instance, any replica can handle any request at any time — no warm-up, no session migration.
- **Detect Faults via Docker Provider Event Stream:** Traefik's Docker provider ( `--providers.docker=true`, `infrastructure/docker-compose.yml` ) subscribes to Docker lifecycle events. When a replica container exits, Traefik receives the event from the Docker API via `docker-api-compat` → `docker-socket-proxy` and immediately removes that container's IP from the `business@docker` routing table — with no file edit and no gateway restart. This is the detection mechanism verified in `tests/replication/hot/run-test.sh`, which asserts `0 failures` across 15 consecutive requests issued at 1 req/s after a `docker stop`.
- **Removal from Service (Health-Check-Based Exclusion):** The service labels in `docker-compose.yml` configure an active health check: `healthcheck.path=/api/health`, `interval=10s`, `timeout=3s`. This acts as a reactive exclusion gate: if a replica becomes unresponsive before the container exit event is processed, Traefik stops forwarding requests to it. This is an exclusion mechanism, not an admission gate — replicas are added to the pool on container start, not after a first successful probe.
- **Automatic Restart (Fault Recovery):** `blume-business-logic-ms` is configured with `restart: unless-stopped` and `deploy.restart_policy: condition: any, delay: 5s`. Docker automatically restarts the failed container after the 5-second delay, initiating the path back to full redundancy without operator action.

## Applied architectural patterns

**Active Redundancy (Active/Active Hot Spare):** Three stateless replicas of `blume-business-logic-ms` run concurrently behind Traefik's Docker provider in an Active/Active configuration. Statefulness is entirely externalized: persistent data lives in the shared `mysql` instance and authentication is carried in the JWT on every request ( `JWT_SECRET` injected as environment variable, no server-side session). When one replica fails, Traefik removes its IP from `business@docker` via the Docker event stream; the two surviving replicas absorb the full load instantaneously. The test in `tests/replication/hot/run-test.sh` codifies the response measure: `docker stop` on one of three running replicas must produce exactly 0 failed requests across 15 consecutive calls.  $RTO \approx 0$ ;  $RPO = 0$  for the application tier.

## Warm Replication

### Scenario

Loss of the primary database node with a synchronized standby ready for promotion.

### SOURCE

Failure of the primary MySQL server (`mysql`) or planned maintenance need.

### STIMULUS

Continuous writes to `blume_business_db` (primary MySQL) while `mysql-warm` receives asynchronous binlog replication.

## ARTIFACT

MySQL 8.4 Primary/Standby pair: primary with binlog (mysql) + read-only replica (mysql-warm) synchronized via CHANGE REPLICATION SOURCE TO. The application still points only to the primary (DB\_HOST=mysql).

## ENVIRONMENT

Normal operation; overlay docker-compose.replication.yml with replication profile; internal blume\_data network.

## RESPONSE

The warm replica applies binlog events in near real time; it remains inactive for application traffic until manual promotion (STOP REPLICATION; SET GLOBAL read\_only=0; + DB\_HOST change).

## RESPONSE MEASURE

(1) Replica\_IO\_Running = Yes and Replica\_SQL\_Running = Yes. (2) Seconds\_Behind\_Source  $\leq$  5 s. (3) Data inserted in the primary visible in warm within  $\leq$  15 s. (4) @@read\_only = 1 on warm. RPO  $\approx$  replication lag (seconds); RTO  $\approx$  minutes (promotion + reconfiguration).

```
=====
REPLICATION WARM – MySQL standby sincronizado (mysql-warm)
=====

PASS: Replica_IO_Running = Yes (got: Yes)
PASS: Replica_SQL_Running = Yes (got: Yes)
PASS: Lag de replicación $\leq$ 5 s (got: 0s)
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
PASS: Marcador replicado a mysql-warm
PASS: Tiempo de propagación $\leq$ 15 s
PASS: Réplica warm permanece read-only (no sirve escrituras de app) (got: 1)

=====
WARM REPLICATION
Results: 6/6 PASS | 0 FAIL
=====
```

## Applied architectural tactics

- **Maintain a Passive Standby Copy (Warm Spare):** mysql-warm runs continuously as a MySQL 8.4 instance configured with --server-id=2, --read-only=1, --relay-log=relay-bin, and --skip-replica-start (all in docker-compose.yml). It receives and applies all writes from the primary mysql (--server-id=1, --log-bin=mysql-bin, --binlog-format=ROW, --sync-binlog=1) via asynchronous binlog replication. The --read-only=1 flag is enforced at the MySQL server level, preventing the warm replica from accidentally accepting application writes during normal operation — verified by warm/run-test.sh asserting @@read\_only = 1.
- **Monitor and Auto-Repair Replication (Continuous Watchdog):** The replication-warm-setup sidecar in docker-compose.yml runs warm-watchdog.sh as its entrypoint. Every WARM\_CHECK\_INTERVAL seconds (default: 60, configurable via env var), the watchdog queries SHOW REPLICA STATUS\G and checks that both Replica\_IO\_Running and Replica\_SQL\_Running are Yes. If either thread has stopped, it automatically invokes warm-replicate-core.sh to re-establish the replication link — no operator action required for transient replication failures.
- **State Resynchronization (Binlog Streaming):** warm-replicate-core.sh bootstraps the standby by: (1) reading SHOW BINARY LOG STATUS on the primary; (2) cloning the initial dataset via mysqldump --single-transaction if the warm instance is empty; (3) issuing CHANGE REPLICATION SOURCE TO with the exact binlog file and position captured after the clone; and (4) executing START REPLICATION. Subsequent changes stream as ROW-format binlog events, keeping Seconds\_Behind\_Source within the  $\leq$  5 s threshold tested in warm/run-test.sh.
- **Manual Reconfiguration on Failover:** promote-warm.sh encapsulates the promotion procedure: STOP REPLICATION; RESET REPLICATION ALL; SET GLOBAL read\_only = 0. After running it, the operator must update DB\_HOST in docker-compose.yml (or the .env override) and run docker compose up -d blume-business-



logic-ms . This manual procedure is the source of the minutes-scale RTO — explicitly documented in the script's output as "RTO estimado: minutos" .

## Applied architectural patterns

**Passive Redundancy (Warm Spare):** mysql-warm maintains a continuously synchronized copy of blume\_business\_db via asynchronous MySQL binlog replication, but serves zero application traffic during normal operation. It is a standby, not a load-sharing peer. The replication link is bootstrapped and auto-repaired by the replication-warm-setup sidecar (running warm-watchdog.sh + warm-replicate-core.sh ), so the warm copy stays current without operator attention. Promotion to primary is a deliberate manual act — promote-warm.sh removes the read-only constraint and a subsequent config change redirects the application — trading automatic failover for simplicity: no split-brain risk, no fencing protocol, no quorum. RPO is bounded by the replication lag ( Seconds\_Behind\_Source  $\leq$  5 s ); RTO is on the order of minutes.

---

## Cold Replication

### Scenario

Total disaster on the primary MySQL of blume\_record\_db with no standby online; recovery from offline backup.

### SOURCE

Data corruption, accidental deletion, or unrecoverable failure of the recordings-mysql container/volume.

### STIMULUS

DROP DATABASE recordings simulating total loss of the primary; data written after the last backup falls outside the recovery scope.

### ARTIFACT

Periodic logical backup ( mysqldump ) of blume\_record\_db stored in replication/backups/ ; cold instance = no active service until restore is executed. Automated by the mysql-cold-backup sidecar (interval: 1 h, retention: 24 backups).

### ENVIRONMENT

Normal operation until the disaster; backups generated automatically by mysql-cold-backup or on demand via backup-cold.sh ; Docker Compose stack.

### RESPONSE

Manual procedure: (1) backup-cold.sh , (2) upon disaster restore-cold.sh <backup.sql> , (3) restart of dependent microservices ( blume\_record\_ms ).

### RESPONSE MEASURE

(1) Data prior to the backup recovered ( marker\_before = 1 ). (2) Data written after the backup lost ( marker\_after = 0 ) — demonstrates RPO. (3) GET /api/recordings returns HTTP 200 after restore. (4) RTO  $\leq$  120 s (measured restore time). RPO = interval between backups (default: 1 h).

---

#### REPLICATION COLD — Recuperación desde backup (mysqldump)

---

```
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysqldump: [Warning] Using a password on the command line interface can be insecure.
PASS: Respaldo cold generado (/home/turing/Documents/UNAL/1C/infrastructure/replication/backups/blume-20260607-131117.sql)
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
PASS: Dato post-backup presente antes del desastre (got: 1)

Simulando desastre: DROP DATABASE blume...
mysql: [Warning] Using a password on the command line interface can be insecure.
PASS: Servicio degradado/no disponible tras desastre (got HTTP 500)
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
PASS: Marcador pre-backup recuperado (RPO = punto del dump) (got: 1)
PASS: Marcador post-backup NO recuperado (pérdida acorde a RPO cold) (got: 0)
PASS: Servicio restaurado tras recovery (got: 200)
PASS: RTO de restauración $\leq$ 120 s (got: 2032 ms)
```

---

COLD REPLICATION  
Results: 8/8 PASS | 0 FAIL

---

## Applied architectural tactics

- **State Capture via Periodic Logical Backup:** The `mysql-cold-backup` sidecar ( `docker-compose.yml` , `entrypoint: cold-backup-loop.sh` ) runs continuously and invokes `mysqldump --single-transaction --routines --triggers` against `recordings-mysql` every `BACKUP_INTERVAL_SECONDS` (default: 3600 , configurable via `COLD_BACKUP_INTERVAL_SECONDS` ). `--single-transaction` guarantees a consistent InnoDB snapshot without locking the tables during the dump. Backups are written to `replication/backups/` and retention is managed by the loop itself: it keeps the last `BACKUP_RETENTION_COUNT` files (default: 24 ) and purges older ones with `ls -lt | tail -n | xargs rm` .
- **Rollback to a Known-Good Checkpoint:** `restore-cold.sh` takes a `.sql` backup file as its sole argument and executes: `DROP DATABASE IF EXISTS + CREATE DATABASE + mysql < backup` . This deliberately rolls the database back to the exact state captured at backup time. The test in `cold/run-test.sh` explicitly asserts this behavior: a marker inserted before the backup ( `MARKER_BEFORE` ) is recovered; a marker inserted after the backup ( `MARKER_AFTER` ) is permanently lost — demonstrating both the recovery capability and the RPO boundary.
- **No Active or Passive Standby (Cold Spare):** There is no `recordings-mysql-warm` or second container in `docker-compose.yml` for `recordings-mysql` . Recovery requires restoring into the existing `recordings-mysql` container from the `.sql` file. The absence of a live standby is a deliberate tradeoff appropriate for `blume_record_db` : recording metadata has lower availability requirements than the primary business database, so the simpler cold-backup strategy suffices.

## Applied architectural patterns

**Cold Spare (Offline Backup and Restore):** `blume_record_db` has no active or passive standby. The `mysql-cold-backup` sidecar captures hourly logical snapshots of `recordings-mysql` via `mysqldump` and retains the last 24 backups. Recovery is entirely manual: `backup-cold.sh` (on-demand one-off snapshot) followed by `restore-cold.sh <backup.sql>` (DROP + recreate + reimport). The test in `cold/run-test.sh` measures and asserts the two key SLOs:  $RTO \leq 120\,000$  ms (measured from `DROP DATABASE` through `restore-cold.sh` completion) and  $RPO$  = the age of the last backup (data written after the last dump is permanently lost, as confirmed by the `MARKER_AFTER = 0` assertion). This is the highest-RPO and highest-RTO tier of the three strategies, and is justified by the lower availability requirements of recording metadata relative to authentication and live-streaming state.

## Cluster

### Scenario

#### Source

The `blume_stream_activities_ms` microservice

#### Stimulus

A node on the `blume_stream_activities_ms` microservice cluster fails on a logical or physical level due to a OS level error, hardware issues or similar and stops responding to requests

#### Artifact

The `blume_stream_activities_ms` elixir-based cluster with `libcluster`

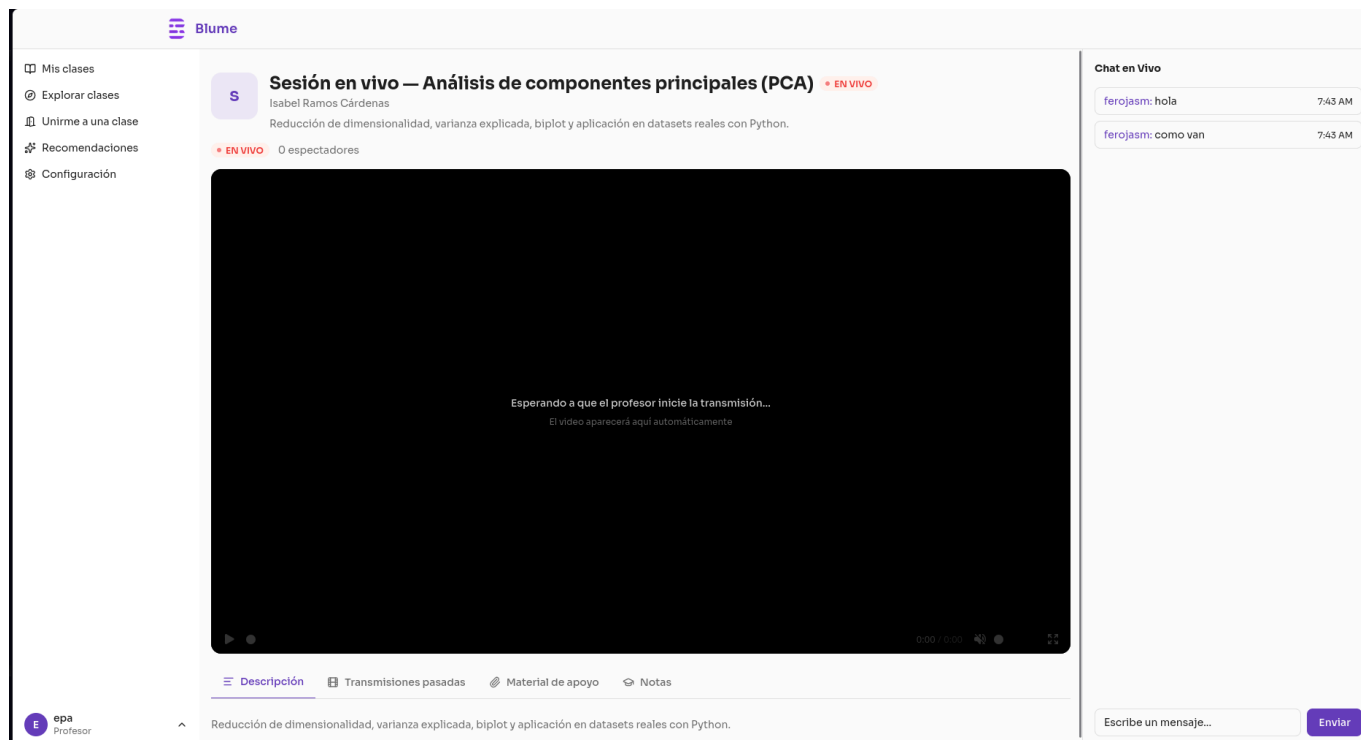
#### Environment

System deployed, `stream_activities` cluster deployed over multiple machines

#### Response

Traefik stops getting the health check from one of the nodes and proceeds to route all current and incoming connections to the healthy nodes, the system proceeds with normal operations

As shown in this example:



The chat operating normally

```
Container blume-blume_stream_activities_ms-1 Stopping
blume_stream_activities_ms-1 | 02:37:01.515 [notice] SIGTERM received - shutting down
```

We simulate a physical failure by shutting down the node thats currently asigned to this user

```
blume_stream_activities_ms-1 | 02:37:01.400 [info] CONNECTED TO StreamActivitiesWeb.UserSocket in 5ms
```

The user gets asigned a new node in aproximately 500ms (on the screenshots it would look like the user conected before the container shut down but thats because of small differences on the system clock of both systems) with less than a second of noticable delay when sending a message at the same time as the container was shut down



A user sending a message at the same time as the node was being shut down (the message only shows up on chat if it was properly received by the stream\_activities backend)

## Response Measure

- The failing of one node doesn't stop the system from operating normally
- The downtime recorded for a user that was currently connected to the microservice is of less than a second
- The stateful database maintains integrity
- Architectural Tactics applied:
  - Sanity Checking: The system through Traefik does regular sanity checks (every 10s) before routing any traffic to a specific node, ensuring that the cluster is operating correctly
  - Reconfiguration: The system moves all current users of the node that fails to the rest of the cluster ensuring normal operations
  - Redundant Spare: Since multiple physical machines are running the availability is ensured over a hardware/logical failure of one or multiple machines

## Circuit Breaker

### Scenario

#### SOURCE

One of the `blume_business_logic_ms` replicas has failed due to sustained load.

#### STIMULUS

Due to the sustained load, the replica that failed returns HTTP 500 or timeout after requests start taking longer than 30 seconds for a sustained window (more than 10 consecutive calls). The website is no longer responsive to some users, because the remaining replicas can't service the missing requests.

#### ARTIFACT

We implemented Circuit Breaker at two levels. First, we implement it at `blume_business_logic_ms` router, using *Traefik* pre-built Circuit Breaker functionality. We also implement a Circuit Breaker on the `blume_recomendations_ms` microservice, using the pre-built functionality offered by the `circuitbreaker` Python library. Furthermore, we implemented a third level of Circuit Breaker internally within `blume_business_logic_ms` using the Resilience4j library to protect the database connection pool from cascading failures.

#### ENVIRONMENT

Normal operations. Docker Compose launches `blume_business_logic_ms` with 3 replicas and the Load Balancer implemented on the previous Artifact. Also, we use *Toxiproxy* to simulate backend failures, to force Circuit Breaker to change states, and measure the changes on the system's functionality.

#### RESPONSE

For testing purpose, we will use the "Explorar" endpoint to test the baseline functionality. We simulate this with the following endpoint loop:

```
while true; do
 CODE=$(curl -s -o /dev/null -w "%{http_code}" \
 https://localhost/api/cursos/explorar)
 echo "$(date +%H:%M:%S) explorar → $CODE"
 sleep 1
done
```

```

jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$ while true; do
 CODE=$(curl -s -o /dev/null -w "%{http_code}" \
 https://localhost/api/cursos/explorar)
 echo "$(date +%H:%M:%S) explorar → $CODE"
 sleep 1
done
18:57:06 explorar → 200
18:57:08 explorar → 200
18:57:09 explorar → 200
18:57:10 explorar → 200
18:57:11 explorar → 200
18:57:12 explorar → 200
18:57:14 explorar → 200
18:57:15 explorar → 200
18:57:16 explorar → 200
18:57:17 explorar → 200
18:57:18 explorar → 200
18:57:19 explorar → 200
18:57:20 explorar → 200
18:57:21 explorar → 200
18:57:22 explorar → 200
18:57:23 explorar → 200
18:57:24 explorar → 200
18:57:25 explorar → 200
18:57:26 explorar → 200
18:57:27 explorar → 200
18:57:28 explorar → 200
18:57:29 explorar → 200
18:57:30 explorar → 200
18:57:31 explorar → 200
18:57:32 explorar → 200
18:57:33 explorar → 200
18:57:34 explorar → 200
18:57:35 explorar → 200
18:57:36 explorar → 200

```

First, we can confirm that the endpoint is functional under the standard deployment. Now, we simulate the failure of one of the replicas, using the following commands:

```

toxiproxy-cli --host localhost:8474 toxic add \
 -t latency -a latency=5000 business-logic-proxy

```

```

toxiproxy-cli --host localhost:8474 toxic add \
 -t reset_peer business-logic-proxy

```

We use *Toxiproxy* framework in order to simulate the failures. With the first command, we simulate delays, as a prelude of high workload. With the second command, we simulate timeouts. Here, we can see the loop starting to return 502 error codes, instead of 200. Since we haven't implemented the architectural pattern yet, there is no recovery strategy for this failure, so it just starts failing. The only way to resume operation is with *Toxiproxy*, but the system is unable to do so on their own.

```
jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$ toxiproxy-cli --host localhost:8474 toxic add \
-t latency -a latency=5000 business-logic-proxy
Added downstream latency toxic 'latency_downstream' on proxy 'business-logic-proxy'
jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$ toxiproxy-cli --host localhost:8474 toxic add \
-t reset_peer business-logic-proxy
Added downstream reset_peer toxic 'reset_peer_downstream' on proxy 'business-logic-proxy'
jujgomezru@DESKTOP-T5D8MHC:~/unal/arquitecturaDeSoftware/infrastructure$
```

```
19:29:45 explorar → 200
19:29:51 explorar → 200
19:29:57 explorar → 200
19:30:03 explorar → 200
19:30:14 explorar → 502
19:30:20 explorar → 502
19:30:26 explorar → 502
19:30:32 explorar → 502
19:30:38 explorar → 502
19:30:44 explorar → 502
19:30:50 explorar → 502
19:30:56 explorar → 502
19:31:02 explorar → 502
19:31:09 explorar → 502
19:31:15 explorar → 502
19:31:21 explorar → 502
19:31:27 explorar → 502
19:31:33 explorar → 502
█
```

After we implement the Circuit Breaker, when we apply the same simulated failures on the backend, instead of returning 502, it returns 503 error codes. 503 error codes are faster, since it doesn't require a timeout. Also, the Circuit Breaker actively changes the response of the endpoint if it recognizes the frontend is failing. As we can see on the frontend images, when `/explorar` is called, it shows an error message on the frontend, showing it captures the exception correctly.

20:45:44 explorar → 200  
20:45:45 explorar → 200  
20:45:46 explorar → 200  
20:45:47 explorar → 200  
20:45:48 explorar → 200  
20:45:49 explorar → 200  
20:45:50 explorar → 200  
20:45:51 explorar → 200  
20:45:53 explorar → 200  
20:45:54 explorar → 200  
20:45:55 explorar → 200  
20:45:56 explorar → 200  
20:45:57 explorar → 200  
20:45:58 explorar → 200  
20:46:04 explorar → 200  
20:46:10 explorar → 200  
20:46:16 explorar → 200  
20:46:22 explorar → 200  
20:46:32 explorar → 502  
20:46:33 explorar → 503  
20:46:34 explorar → 503  
20:46:35 explorar → 503  
20:46:36 explorar → 503  
20:46:37 explorar → 503  
20:46:38 explorar → 503  
20:46:39 explorar → 503  
20:46:41 explorar → 503  
20:46:42 explorar → 503  
20:46:43 explorar → 503  
20:46:44 explorar → 503  
20:46:45 explorar → 503  
20:46:46 explorar → 503  
20:46:47 explorar → 503  
20:46:48 explorar → 503  
20:46:49 explorar → 503

# Bienvenido de nuevo

Ingresa a tu cuenta para continuar aprendiendo

Tu correo

Contraseña



Ingresar

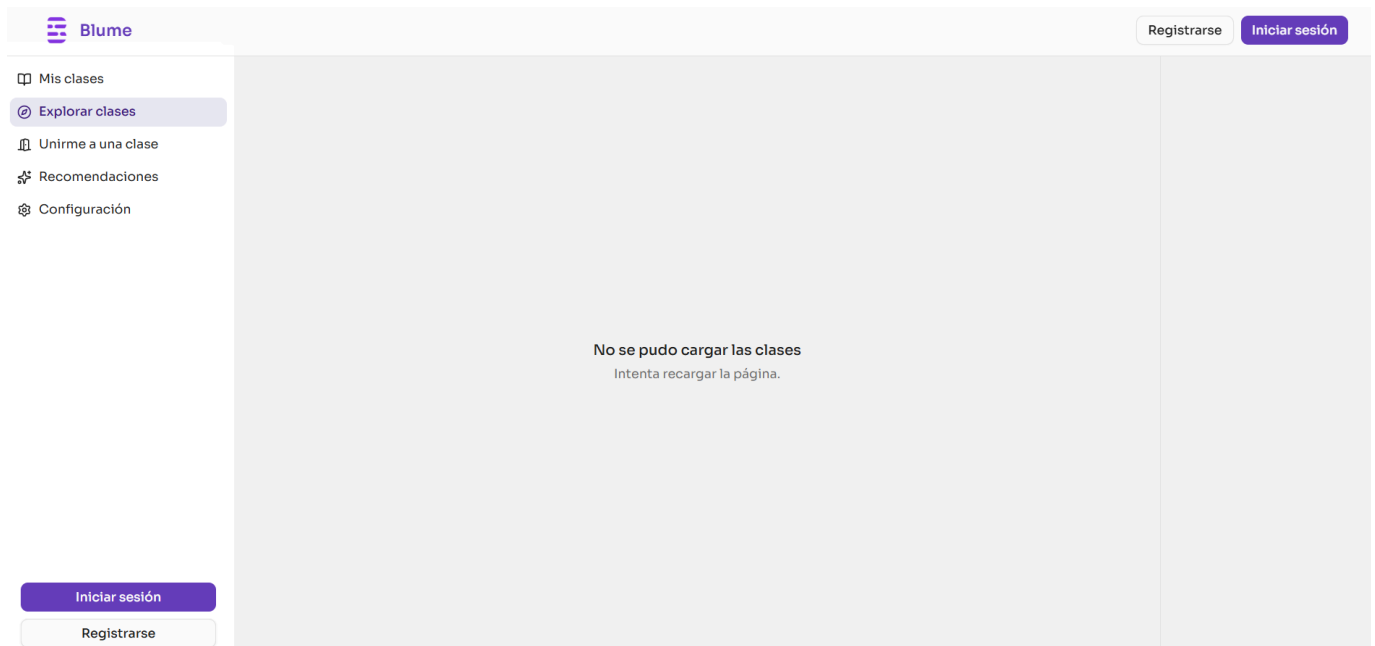


 Continuar con Google

HTTP 503

¿No tienes cuenta? [Crear una cuenta](#)





Also, the system is able to restore to 200 codes by itself, once the issue has been solved. When the Circuit Breaker recognizes the backend is functional again, the failing endpoint starts working again as intended, rather than having to be deployed manually again.

```
20:51:38 explorar → 503
20:51:39 explorar → 503
20:51:40 explorar → 503
20:51:41 explorar → 503
20:51:42 explorar → 503
20:51:43 explorar → 503
20:51:44 explorar → 503
20:51:45 explorar → 503
20:51:47 explorar → 503
20:51:53 explorar → 200
20:51:59 explorar → 200
20:52:05 explorar → 200
20:52:11 explorar → 200
20:52:17 explorar → 200
20:52:23 explorar → 200
20:52:29 explorar → 200
20:52:35 explorar → 200
20:52:41 explorar → 200
```

## RESPONSE MEASURE

In order to document the Response Measure of the Circuit Breaker, we created a Python script ( `infrastructure/tests/performance/cb_verification.py` ) tasked with sending requests without and with *Toxiproxy* commands, in order to determine if the system changes its functionality, depending on the service status of `blume_business_logic_ms` . Here, we show the results of the script. We include first the results of the script on an earlier version of the project, with no Circuit Breaker implementation:

## Circuit Breaker Verification Report

Configuration:

- Rate: 2 req/s × 2 endpoints
- Phases: baseline=15s | fault=70s | recovery=55s
- CB: checkPeriod=10s | fallbackDuration=30s | recoveryDuration=10s

Per-Phase Summary

| Endpoint        | Phase    | n   | Err%   | Avg  | P99  | Codes      |
|-----------------|----------|-----|--------|------|------|------------|
| explorar        | BASELINE | 30  | 0.0%   | 11ms | 32ms | {200: 30}  |
| explorar        | FAULT    | 140 | 100.0% | 15ms | 44ms | {502: 140} |
| explorar        | RECOVERY | 110 | 0.0%   | 10ms | 33ms | {200: 110} |
| recommendations | BASELINE | 30  | 0.0%   | 9ms  | 30ms | {200: 30}  |
| recommendations | FAULT    | 140 | 0.0%   | 9ms  | 12ms | {200: 140} |
| recommendations | RECOVERY | 110 | 0.0%   | 11ms | 53ms | {200: 110} |

Measure 1 — Error rate < 5% within one checkPeriod (10s)

Endpoint: /recommendations (has TTL cache fallback)

| Window | Err% | n  | Status             |
|--------|------|----|--------------------|
| +0s    | 0.0% | 20 |                    |
| +10s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +20s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +30s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +40s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +50s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +60s   | 0.0% | 20 | FALLBACK SERVING ✓ |

Result: PASS ✓ — error rate < 5% at +10s into fault phase

Measure 2 — P99 latency < 500ms during degraded period

Endpoint: /explorar (no fallback — shows raw CB fast-fail effect)

| State                            | P99  | Result |
|----------------------------------|------|--------|
| Before CB trips (502/ERR, n=140) | 44ms | —      |

Measure 3 — Full recovery ≤ 40s without operator intervention

Endpoint: /explorar

| Event | Time |
|-------|------|
|-------|------|

|               |          |
|---------------|----------|
| Fault removed | 14:28:05 |
| First 200     | 14:28:05 |
| Recovery time | 0.1s     |

**Result: PASS** ✓ — 0.1s (target ≤ 40s)

### Measure 4 — Zero manual restarts required

**Result: PASS** ✓ — Script never invoked `docker restart` or any container command. Recovery was entirely driven by the Traefik CB state machine.

Now, we show the results of the script on the current version, which includes the Circuit Breaker implementation.

## Circuit Breaker Verification Report

**Configuration:**

- Rate: 2 req/s × 2 endpoints
- Phases: baseline=15s | fault=70s | recovery=55s
- CB: checkPeriod=10s | fallbackDuration=30s | recoveryDuration=10s

### Per-Phase Summary

| Endpoint        | Phase    | n   | Err%   | Avg  | P99  | Codes               |
|-----------------|----------|-----|--------|------|------|---------------------|
| explorar        | BASELINE | 30  | 0.0%   | 11ms | 25ms | {200: 30}           |
| explorar        | FAULT    | 140 | 100.0% | 6ms  | 55ms | {502: 12, 503: 128} |
| explorar        | RECOVERY | 110 | 7.3%   | 11ms | 48ms | {503: 8, 200: 102}  |
| recommendations | BASELINE | 30  | 0.0%   | 8ms  | 22ms | {200: 30}           |
| recommendations | FAULT    | 140 | 0.0%   | 9ms  | 12ms | {200: 140}          |
| recommendations | RECOVERY | 110 | 0.0%   | 9ms  | 12ms | {200: 110}          |

### Measure 1 — Error rate < 5% within one checkPeriod (10s)

**Endpoint:** `/recommendations` (has TTL cache fallback)

| Window | Err% | n  | Status             |
|--------|------|----|--------------------|
| +0s    | 0.0% | 20 |                    |
| +10s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +20s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +30s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +40s   | 0.0% | 20 | FALLBACK SERVING ✓ |
| +50s   | 0.0% | 20 | FALLBACK SERVING ✓ |

|      |      |    |                    |
|------|------|----|--------------------|
| +60s | 0.0% | 20 | FALLBACK SERVING ✓ |
|------|------|----|--------------------|

**Result: PASS** ✓ — error rate < 5% at +10s into fault phase

---

## Measure 2 — P99 latency < 500ms during degraded period

**Endpoint:** `/explorar` (no fallback — shows raw CB fast-fail effect)

| State                           | P99  | Result                  |
|---------------------------------|------|-------------------------|
| Before CB trips (502/ERR, n=12) | 58ms | —                       |
| After CB trips (503, n=128)     | 8ms  | PASS ✓ (target < 500ms) |

Fast-fail speedup: **7.4x** faster after CB opens

---

## Measure 3 — Full recovery ≤ 40s without operator intervention

**Endpoint:** `/explorar`

| Event         | Time     |
|---------------|----------|
| Fault removed | 11:44:25 |
| First 200     | 11:44:27 |
| Recovery time | 1.6s     |

**Result: PASS** ✓ — 1.6s (target ≤ 40s)

---

## Measure 4 — Zero manual restarts required

**Result: PASS** ✓ — Script never invoked `docker restart` or any container command. Recovery was entirely driven by the Traefik CB state machine.

---

These reports show the tradeoff that we need with the implementation of the Circuit Breaker Pattern. Measure 2 is the most revealing one, since switching the exception handling strategy from 502 to 503, significantly decreases the response time of the failing service by 7.4 timefold (58ms vs. 8ms). Not only the handling of the failure is faster, it's also more stable, because it just prevents the service from being reachable rather than showing an exception. Measure 3, on the otherhand, actually increase in time compared to the version before the Circuit Breaker implementation (1.6 seconds vs. 0.1). The question the architect (us) has to make is, which speedup is more important. Measure 3 shows how long does the service become fully reachable after it returns to normal operations. Before Circuit Breaker this is quicker since it keeps sending timed out requests until it finally works, whereas the Circuit Breaker forces a time window before it attempts to connect in a Half-Open state. This waiting period is only relevant at the moment of connection, whereas Measure 2 is relevant during the failure. There might be expectations, but in this case, since failure tend to be prolonged and require manual intervention, it's significantly more time consuming to wait for timeout requests during the entire offline period, than waiting a bit longer when it finally resumes traffic. Therefore, Measure 3 proves the Circuit Breaker is always a counterbalance, but in our example, the benefits (Measure 2) clearly outweigh the costs (Measure 3). Measure 1 and Measure 4 are just regular health checks to the backend, showing that in real life, there were no uncontrolled failures during testing, and the system was able to connect back to the frontend once this resumed operations.

### Architectural Tactics

#### DETECT FAULTS

- **Monitor:** *Traefik* itself its a continuous monitor. On normal function, *Traefik* observes every request that passes through `blume_business_logic_ms` and tracks its response and error codes in real time.

- **Condition monitoring:** The Circuit Breaker currently observes two conditions to trip up. It checks for a `ResponseCodeRatio(500,600,0,600) > 0.30` , i.e. requests with 500 or 600 error codes, compared with every response code, and is configured to open the circuit if more than 30% or requests consist of error codes, and `NetworkErrorRatio() > 0.10` i.e. more than 10% or requests with an error code on the latest time window. If any of these conditions is fulfilled, the Circuit trips to an Open state.
- **Exception Detection:** As mentioned before, the Circuit Breaker detects every requests' code, both success and error codes, and uses these error codes for Condition Monitoring.

#### PREPARATION AND REPAIR

- **Exception Handling:** When the Circuit Breaker is in Open State, it avoids returning any actual exception to the User. Instead, it automatically returns a 503 error code, indicating the service is not available.
- **Graceful Degradation:** The 503 error code returned by the Circuit Breaker is a default error view, rather than a compiling error or a timeout. While the service is not available, the error screen is decisively more stable. Also, when `blume_recomendations_ms` is not available, it uses its TTL Cache as a backup, providing a supplementary version of its service in the meantime.
- **Reconfiguration:** Circuit Breaker changes dynamically the routing of *Traefik* depending on its inner state (Open, Closed, Half-Open), which configures itself on its own, based of its internal rules.

#### PREVENT FAULTS

- **Removal from service:** When the Circuit Breaker is open, the defective replica that caused the Circuit to open is temporarily removed from service, no further requests are sent to it until it returns to Half-Open.
- **Exception Prevention:** Since the Circuit Breaker avoids sending any further requests to the defective replica in the meantime, it actively avoids sending requests that are knowingly going to fail.

#### Architectural Pattern

We used the pre-configured Circuit Breaker functionality from *Traefik*, which is defined under the `infrastructure/traefik/dynamic.yml` file as follows:

```
http:
 middlewares:
 hsts:
 headers:
 stsSeconds: 15768000
 stsIncludeSubdomains: true
 stsPreload: true
 forceSTSHeader: true

 business-cb:
 circuitBreaker:
 expression: "ResponseCodeRatio(500, 600, 0, 600) > 0.30 || NetworkErrorRatio() > 0.10"
 checkPeriod: 10s
 fallbackDuration: 30s
 recoveryDuration: 10s
```

Here, we define a middleware in our *Traefik* instance, called `business-cb` that acts as our Circuit Breaker. Here, we define the Middleware as a `circuitBreaker` type, and we configure the Circuit. As mentioned before, `expression` determines when does the circuit open. The `checkPeriod` parameter defines the length of the window (it monitors the requests on a 10 second window, and applies the condition). The `fallbackDuration` parameter determines how long does it take for the Open Circuit to automatically shift to Half-Open, and the `recoveryDuration` parameter determines for how long does the Circuit remains Half-Open before determining if it Closes or goes back to Open.

```
business:
 rule: "PathPrefix(`/api`)"
 entryPoints: ["websecure"]
 middlewares: ["hsts", "business-cb"]
 priority: 10
 service: business-proxy
 tls: {}
```

Also, when defining the routers, the `business` router uses both the standard `hsts` middleware (that every system router uses), and the newly defined Circuit Breaker `business-cb`. Here, we defined all requests sent to any of the `blume_business_logic_ms` will be monitored by our Circuit Breaker.

We also defined the *Toxiproxy* configuration on `infrastructure/toxiproxy/toxiproxy.json` as follows:

```
[
 {
 "name": "business-logic-proxy",
 "listen": "0.0.0.0:38082",
 "upstream": "blume-business-logic-ms:8082",
 "enabled": true,
 "toxics": []
 }
]
```

Our new `docker-compose.yml` introduces *Toxiproxy* on the deployment, so it needs a default configuration for it to run. The testing script used for the previous reports changes the *Toxiproxy* configuration to introduce `toxics`. The toxics are the simulated network failures to test the real-life functionality of the Circuit Breaker.

The Python testing script defines the following methods:

```
@dataclass
class Sample:
 ts: float # epoch seconds
 phase: str # BASELINE | FAULT | RECOVERY
 endpoint: str # explorar | recommendations
 code: int # HTTP status; 0 = connection/timeout error
 latency_ms: float

results: list[Sample] = []
results_lock: threading.Lock = threading.Lock()

— Toxiproxy helpers —————

def _toxiproxy(method: str, path: str, body: Optional[dict] = None):
 url = f"{TOXIPROXY_API}{path}"
 data = json.dumps(body).encode() if body else None
 req = urllib.request.Request(url, data=data, method=method)
 if body:
 req.add_header("Content-Type", "application/json")
 try:
 with urllib.request.urlopen(req, timeout=5) as resp:
 return json.loads(resp.read())
 except Exception:
 return None

def inject_fault():
 _toxiproxy("DELETE", f"/proxies/{PROXY_NAME}/toxics/{TOXIC_NAME}")
 _toxiproxy("POST", f"/proxies/{PROXY_NAME}/toxics", {
 "name": TOXIC_NAME,
 "type": "reset_peer",
 "stream": "downstream",
 "toxicity": 1.0,
 "attributes": {},
 })
```

```

def remove_fault():
 _toxiproxy("DELETE", f"/proxies/{PROXY_NAME}/toxics/{TOXIC_NAME}")

— HTTP helper —————

def build_session() -> requests.Session:
 session = requests.Session()
 session.verify = CA_CERT
 # No retries – every failure must be recorded as-is for the CB test
 adapter = HTTPAdapter(max_retries=Retry(total=0))
 session.mount("https://", adapter)
 session.mount("http://", adapter)
 return session

def do_request(url: str, session: requests.Session) -> tuple[int, float]:
 start = time.monotonic()
 try:
 resp = session.get(url, timeout=5)
 return resp.status_code, (time.monotonic() - start) * 1000
 except requests.exceptions.HTTPError as e:
 return e.response.status_code, (time.monotonic() - start) * 1000
 except Exception:
 return 0, (time.monotonic() - start) * 1000

— Worker —————

def worker(
 label: str,
 url: str,
 session: requests.Session,
 phase_events: list[tuple[float, str]],
 stop_event: threading.Event,
):
 interval = 1.0 / RATE_RPS
 while not stop_event.is_set():
 now = time.time()
 phase = "BASELINE"
 for evt_ts, evt_name in phase_events:
 if now >= evt_ts:
 phase = evt_name

 loop_start = time.monotonic()
 code, latency_ms = do_request(url, session)

 sample = Sample(ts=now, phase=phase, endpoint=label,
 code=code, latency_ms=latency_ms)
 with results_lock:
 results.append(sample)

 sym = "✓" if code == 200 else f"✗{code or 'ERR'}"
 print(f" {datetime.now().strftime('%H:%M:%S')} [{label:16s}] "
 f"{sym:<8} {latency_ms:6.0f}ms [{phase}]")

 wait = max(0.0, interval - (time.monotonic() - loop_start))
 if wait > 0:

```

```

 stop_event.wait(timeout=wait)

— Statistics —————

def p99(latencies: list[float]) -> float:
 if not latencies:
 return 0.0
 s = sorted(latencies)
 return s[max(0, math.ceil(0.99 * len(s)) - 1)]

def error_rate_pct(samples: list[Sample]) -> float:
 if not samples:
 return 0.0
 return sum(1 for s in samples if s.code != 200) / len(samples) * 100

def window_stats(
 samples: list[Sample],
 window_s: int,
) -> list[tuple[float, float, int]]:
 """Returns (offset_s, error_rate_pct, n) per time window."""
 if not samples:
 return []
 t0 = samples[0].ts
 buckets: dict[int, list[Sample]] = defaultdict(list)
 for s in samples:
 buckets[int((s.ts - t0) / window_s)].append(s)
 return [
 (b * window_s,
 sum(1 for s in bs if s.code != 200) / len(bs) * 100,
 len(bs))
 for b, bs in sorted(buckets.items())
]

def first_200_after(samples: list[Sample], after_ts: float) -> Optional[Sample]:
 for s in sorted(samples, key=lambda x: x.ts):
 if s.ts >= after_ts and s.code == 200:
 return s
 return None

```

On the `inject_fault` method, use the `reset_peer` failure option offered by *Toxiproxy*, which simulates a timeout from the `business_logic` microservice. This creates timeout requests for our test conditions. We return to normal service with the `remove_fault` method.

The `do_request` method defines which petition we create. And `worker` defines how to carry out the requests for our test, saving every single request, and their final status (SUCCESS or ERROR). In order to create a new report, the developer has to introduce the following command from the `infrastructure` repository:

```
python3 tests/performance/cb_verification.py
```

## Service Discovery:

### Scenario

- **Source:** The container runtime (Docker), executing the restart: unless-stopped policy after a `blume_business_logic_ms` instance exits. Stimulus: The replacement container is assigned a new, dynamically



generated IP address on the `blume_app` network — one absent from any static configuration file.

- **Artifact:** The service registry: the routing table Traefik maintains dynamically via the Docker provider, mapping the logical service name `business` to the current set of registered instance addresses.
- **Environment:** One `blume_business_logic_ms` instance has just failed; a surviving replica continues handling requests; no static configuration holds a record of the replacement's address, leaving the surviving replica as the sole active member of the pool.
- **Response:** Traefik's Docker provider detects the new container via a Docker lifecycle event, reads its dynamically assigned network address from the Docker API, and registers it under the `business` logical service in the routing table — automatically reintroducing the replacement into the active pool and restoring full redundancy. The routing infrastructure self-corrects in response to the topology change with no configuration edit and no gateway restart.
- **Response Measure:** Redundancy is restored within  $\leq 30$  s of the container starting (bounded by Spring Boot's startup time); the active instance pool returns from 1 to 2 members automatically; 0 manual configuration edits or gateway restarts required; the routing table resolves to 0 stale or hardcoded addresses.

### Applied architectural tactics

- **Discover Service (Event-Driven Registry Update):** Traefik runs with `--providers.docker=true` and accesses the Docker daemon through `docker-api-compat` (an NGINX proxy) → `docker-socket-proxy` ( `tecnativa/docker-socket-proxy` image). The socket proxy enforces strictly read-only access — only `CONTAINERS`, `NETWORKS`, `SERVICES`, `TASKS`, `EVENTS`, `INFO`, `VERSION`, and `PING` operations are permitted. Traefik subscribes to Docker container lifecycle events: when Docker starts a replacement `blume_business_logic_ms` container and assigns it a new dynamic IP on the `blume_app` network, Traefik reads that IP via the Docker API and registers it under the logical service `business@docker` in its routing table — with no file edit and no gateway restart.
- **Health-Check-Based Exclusion (not admission):** The service labels in `docker-compose.yml` configure an active health check: `healthcheck.path=/api/health`, `interval=10s`, `timeout=3s`. This mechanism operates as **reactive exclusion**: if a replica stops responding to probes, Traefik removes it from the rotation. It does not act as an admission gate — the newly started container is added to the pool as soon as Traefik detects the start event, before the first probe has been evaluated. The risk window (traffic forwarded to a still-initializing Spring Boot instance) is bounded by the health check interval (10 s) and the application's actual startup time.
- **Automatic Recovery via Restart Policy:** All services are configured with `restart: unless-stopped` in the compose file. When a `blume_business_logic_ms` container fails, Docker restarts it automatically after a 5-second delay (defined in `deploy.restart_policy.delay`). The surviving replica continues handling requests during that interval without any service interruption.
- **Minimal-Privilege Registry Access:** Docker socket access is not direct from Traefik. It flows through `docker-socket-proxy`, which enforces a least-privilege policy: only read operations are allowed. This limits the blast radius if Traefik were compromised — it cannot create, modify, or delete containers through that interface.

### Applied architectural patterns

**Server-Side Service Discovery:** Clients ( `blume_wa`, `blume_ma` ) send all requests to Traefik's stable address. Traefik is the party that queries the registry (Docker API) and resolves which instances of `blume_business_logic_ms` are currently alive and reachable, keeping the routing table updated in real time. Clients have no knowledge of replica IPs and require no changes when the pool grows or shrinks. Discovery logic is fully encapsulated in the infrastructure layer.

**Self-Registration via Label Convention:** Instances of `blume_business_logic_ms` do not call any registration API. They declare themselves through labels in `docker-compose.yml`: `traefik.enable=true` activates discovery, and `traefik.http.services.business.loadbalancer.server.port=8082` tells Traefik which container port to use. The Traefik flag `--providers.docker.exposedByDefault=false` ensures that only containers with an explicit `traefik.enable=true` label are registered, preventing unintended services from being incorporated into the routing pool. Registration is declarative and co-located with the deployment definition, requiring no registration logic in the application code itself.

## Link to repositories

The main repository of the whole Blume system it's the [infrastructure](#) repository, which is part of the [Salon 1C](#) organization where all other repositories are available.

## General structure

```
1C/
├─ infrastructure/ # Compose local, gateway, variables and execution documentation
├─ blume_wa/ # Next.js Frontend
├─ blume_business_logic_ms/ # API Spring Boot (auth + business)
├─ blume_stream_ms/ # API Go (streaming control + MediaMTX hooks)
├─ blume_record_ms/ # Go service (recordings process and catalog)
├─ blume_stream_activities_ms/ # Phoenix/Elixir (chat interactions and real-time operations)
├─ blume_recommendations_ms/ # Recommendations engine
└─ blume_ma/ # Flutter mobile app (Android / iOS)
```

## Clone all repositories in one parent folder

To run everything with a single `docker compose up`, clone all repositories under the same parent directory:

```
mkdir -p 1C && cd 1C
git clone https://github.com/Salon-1C/infrastructure.git
git clone https://github.com/Salon-1C/blume_wa.git
git clone https://github.com/Salon-1C/blume_business_logic_ms.git
git clone https://github.com/Salon-1C/blume_stream_ms.git
git clone https://github.com/Salon-1C/blume_record_ms.git
git clone https://github.com/Salon-1C/blume_stream_activities_ms.git
git clone https://github.com/Salon-1C/blume_recommendations_ms.git
git clone https://github.com/Salon-1C/blume_ma.git
```

Expected folder layout:

```
1C/
├─ infrastructure/
├─ blume_wa/
├─ blume_business_logic_ms/
├─ blume_stream_ms/
├─ blume_record_ms/
├─ blume_stream_activities_ms/
├─ blume_recommendations_ms/
└─ blume_ma/
```

Run with:

```
cd ./1C/infrastructure

docker compose up
```